Fact checking con evidenza evidenziata — testa per frase

Indice

Fact checking con evidenza evidenziata — testa per frase **4**
1. Che cosa si vuole ottenere . . . 4
2. Come funziona, passo per passo . . . 4
   2.1 Che cosa fa già il decision model . . . 4
   2.2 Che cosa aggiungiamo . . . 4
   2.3 Perché ogni pezzo . . . 5
   2.4 Perché il preambolo . . . 5
   2.5 Che cosa si addestra . . . 5
   2.6 Il divisore in frasi fa parte del modello . . . 6
3. I modelli . . . 6
4. Glossario . . . 6
5. Sequenza di lavoro . . . 6
6. Obiettivi . . . 7

Passo 1 — I modelli e gli hook per frase **8**
In breve . . . 8
1. Pacchetti . . . 8
2. L'architettura comune: Qwen3.5 . . . 8
3. Come si esegue ciascun modello . . . 8
4. Trovare i layer . . . 10
5. Hook sui vettori delle frasi . . . 10
6. Hook sull'attenzione . . . 11
7. Che cosa restituisce `evidence/models.py` . . . 11
8. Test . . . 11
9. GPU e spazio . . . 12

Passo 2 — I dati dalle fonti **12**
In breve . . . 12
1. Le fonti . . . 12
   SciFact . . . 13
   WiCE . . . 13
   HoVer con HotpotQA . . . 13
2. Il divisore in frasi . . . 14
3. Preparazione e comando . . . 14
4. Il formato di una riga . . . 15
5. Statistiche dei dataset costruiti . . . 15
6. Controlli . . . 15

Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata **16**
In breve . . . 16
1. Documenti generati con posizioni note . . . 16
2. Etichette ricavate col modello (ablazione) . . . 17
3. Evidenza spostata . . . 17
4. Quante righe . . . 18
5. Controlli . . . 18

Passo 4 — Split e mix di training **18**
In breve . . . 18
1. Regole degli split . . . 18
2. I set . . . 19
3. Il mix `ev_head` . . . 19
4. Comandi . . . 20

Passo 5 — Il prompt con il preambolo **20**
In breve . . . 20
1. Perché serve . . . 20
2. Esempio (KnowLine-4B, schema) . . . 20
3. Dove va il preambolo negli altri modelli . . . 21
4. Scegliere il testo del preambolo . . . 21
5. Riusare il verdetto o riaddestrarlo . . . 22
6. Dove si scrive la scelta . . . 22

Passo 6 — Dalle frasi ai token, e salvataggio dei vettori **22**
In breve . . . 22
1. L'esempio renderizzato . . . 22
2. Il problema: il documento nel prompt non è scritto come l'originale . . . 23
3. Dai token alle frasi . . . 23
4. Documenti lunghi . . . 24
5. Che cosa si salva (fase 1a) . . . 24
6. Comando . . . 25
7. Test . . . 25

Passo 7 — Metodi semplici da battere **25**
In breve . . . 25
1. I metodi . . . 26
2. Come si calcola E-ATT . . . 26
3. Quali frasi si mostrano . . . 26
4. Comandi . . . 26
5. Tabella da compilare (una per modello) . . . 27

Passo 8 — L'architettura della testa e la loss **27**
In breve . . . 27
1. La rete in una figura . . . 27
2. Gli ingressi . . . 28
3. Strato per strato . . . 28
A. Miscela dei layer . . . 28
B. Vettore della frase . . . 28
C. Comunicazione fra le frasi . . . 28
D. Punteggi e soglia . . . 29
Facoltativo: punteggio per coppie (H-PAIR) . . . 29
4. Parametri e costo (misurati) . . . 29
5. Il codice della testa . . . 29
6. La loss . . . 30
6.1 I tipi di riga . . . 30
6.2 Loss di riferimento (H0): per frase, con soglia fissa . . . 30
6.3 Loss con soglia del documento (ZLPR + MIL) . . . 31
6.4 Termine per le coppie (solo H-PAIR) . . . 32
7. La selezione delle frasi . . . 32
8. Test (`tests/test_evidence_head.py`) . . . 32

Passo 9 — Training **32**
In breve . . . 32
1. Fase 1a: il ciclo di training . . . 33
2. La regola di selezione delle frasi . . . 34

3. Che cosa guardare durante il training . . . 35
4. Fase 1b: con il modello in memoria . . . 35
5. Fase 2: training anche del modello . . . 36
6. Test prima di ogni nuova configurazione . . . 36

Passo 10 — Valutazione **36**
In breve . . . 36
1. Su quali righe . . . 36
2. Metriche dell'evidenza . . . 37
3. Metriche del verdetto . . . 38
4. Prove di comportamento . . . 38
5. Latenza . . . 38
6. Comandi . . . 38
7. Report finale (uno per modello) . . . 39

Passo 11 — Esperimenti **39**
In breve . . . 39
Regole . . . 39
La testa di partenza: H0 . . . 39
Esperimenti principali (in quest'ordine) . . . 39
Controlli sulla configurazione migliore . . . 40
Esperimenti da fare solo se serve . . . 40
elephant-1 e gli altri modelli . . . 40
Controlli finali . . . 41
Registro . . . 41

Passo 12 — Server e rilascio **41**
In breve . . . 41
1. Che cosa fa il server per ogni richiesta . . . 41
2. Come gira ogni modello . . . 42
3. La cartella di rilascio . . . 42
4. Il divisore in frasi va rilasciato con il modello . . . 43
5. Controlli prima del rilascio . . . 43
6. Congelamento, test, model card . . . 43

Checklist e problemi frequenti **44**
1. Checklist per passo . . . 44
2. Se succede... . . . 44

Una strada da esplorare: insegnare alla testa quanto ogni frase regge il verdetto **45**
Il problema . . . 45
L'idea . . . 45
Come farlo . . . 46
Quando tenerlo . . . 46
Rischi . . . 46
Perché vale la pena . . . 47

Riepilogo — Architettura, layer da leggere, loss **47**
1. Il quadro . . . 47
2. L'architettura in sintesi . . . 47
3. Da quali layer leggere, perché, e come sceglierli . . . 48
   3.1 Che cosa offre Qwen3.5 . . . 48
   3.2 Il punto di partenza: 19, 23, 27, 31 . . . 48
   3.3 Come sceglierli davvero: la sonda per layer (prima del training) . . . 49
   3.4 Come sceglierli davvero: i pesi della miscela (dopo il training) . . . 49
   3.5 Le feature di attenzione: da quali layer e teste . . . 50
   3.6 Riepilogo della procedura . . . 50
4. La loss . . . 50

4.1 Che cosa deve insegnare 50
4.2 La loss principale: ZLPR con soglia del documento 50
4.3 La loss di riferimento (H0) 51
4.4 Termine facoltativo per le coppie (H-PAIR) 51
4.5 Pesi e come si combinano 51
4.6 Il rischio da sorvegliare 51

Fact checking con evidenza evidenziata — testa per frase

1. Che cosa si vuole ottenere

Abbiamo un **decision model** fine-tunato come fact checker: riceve un documento e un claim e risponde con P(supportato), in **un solo passaggio del modello sul testo** (un *forward*), senza generare parole. È veloce proprio perché fa un solo forward.

Vogliamo che, quando il claim è supportato, dica anche **quali frasi del documento lo dimostrano**, senza perdere velocità: sempre un solo forward, nessuna parola generata.

Esempio di risposta:

```
{"p_supported": 0.93, "supported": true,
 "evidence": [{"sentence": 0,  "start": 0,    "end": 61,   "score": 0.91},
              {"sentence": 39, "start": 4210, "end": 4262, "score": 0.84}]}
```

- `sentence`: numero della frase nel documento (da 0); `start` / `end`: dove inizia e finisce, in caratteri, nel testo originale.
- Se il claim non è supportato, `evidence` è vuota: non si indica né cosa manca né cosa lo contraddice.

Il caso difficile è l'**evidenza sparsa**: due frasi lontane che solo **insieme** dimostrano il claim.

> Claim: "Nancy R. Howell teaches at a seminary in Kansas City." Frase 1: "Saint Paul School of Theology is a seminary located in Kansas City, Missouri." … 38 frasi che non c'entrano … Frase 40: "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."
>
> Nessuna delle due basta da sola. Evidenza corretta: frasi 1 e 40.

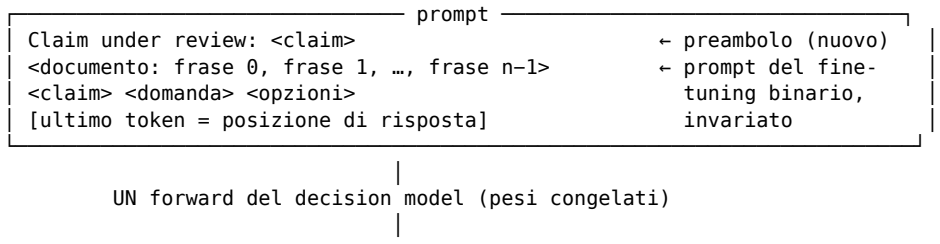
2. Come funziona, passo per passo

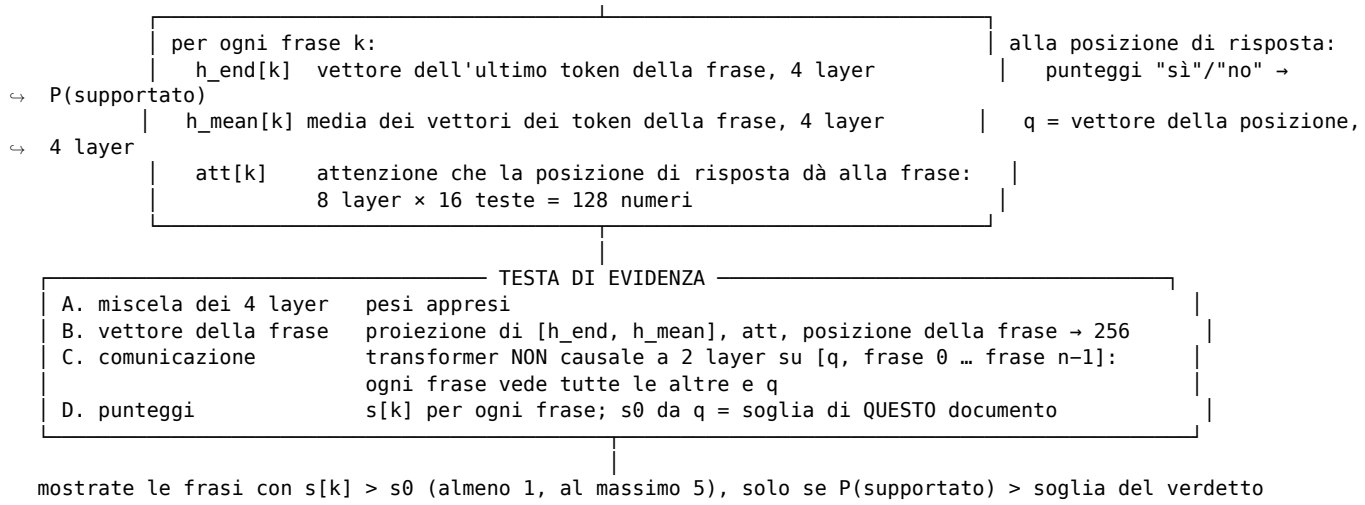
2.1 Che cosa fa già il decision model

Il modello legge il prompt (documento, claim, domanda). Per **ogni token** e per **ogni layer** calcola un vettore di numeri (lo *stato nascosto*): 2.560 numeri nei modelli da 4B, 4.096 in quelli da 9B. Il verdetto si legge dall'**ultimo token del prompt** (la *posizione di risposta*): lì il modello dà un punteggio alla risposta "sì" e uno alla risposta "no", da cui P(supportato). Questa parte esiste già: è il modello del fine-tuning binario, addestrato con una LoRA sulla domanda di fact checking. **Non la tocchiamo.**

2.2 Che cosa aggiungiamo

Il documento si divide in **frasi** con un divisore fisso (`evidence/sentences.py`). Durante il forward si leggono, per ogni frase, alcuni vettori che il modello calcola comunque, e li si dà a una **piccola rete separata**, la *testa di evidenza* (3,7 milioni di parametri contro i 4–9 miliardi del modello), che dà un punteggio a ogni frase.





L'architettura completa, con forme dei tensori, parametri e costo di ogni parte, è nel passo 08. Un riepilogo di architettura, scelta dei layer e loss è in 15.

2.3 Perché ogni pezzo

Pezzo	Perché serve
vettore dell'ultimo token e media della frase	l'ultimo token riassume la frase letta fin lì; la media non dipende dalla punteggiatura finale e pesa tutta la frase
4 layer, non solo l'ultimo	l'ultimo layer è specializzato a predire la risposta; i layer pieni precedenti rappresentano meglio il contenuto delle frasi
attenzione della posizione di risposta	dice su quali frasi il modello si è basato per decidere: un indizio forte, già calcolato
transformer non causale sulle frasi	nel modello la frase 1 non vede la frase 40 (viene dopo); nella testa sì, ed è ciò che risolve l'evidenza sparsa
soglia `s0` del documento	non c'è una soglia unica da tarare; con 3 frasi necessarie la soglia può scendere per farle passare tutte

2.4 Perché il preambolo

Nei modelli **causali** (come KnowLine-4B) ogni token vede solo ciò che lo precede. Nel prompt del fine-tuning binario il claim viene **dopo** il documento: quando il modello legge le frasi non sa ancora cosa deve verificare, e i loro vettori non "parlano" del claim. Mettendo una riga `Claim under review: <claim>` **prima** del documento, ogni frase è letta sapendo cosa si verifica. Il resto del prompt resta identico, compreso il claim dopo il documento, dove il verdetto è abituato a vederlo. Dettagli nel passo 05.

2.5 Che cosa si addestra

Solo la testa. Il modello resta congelato, quindi **il verdetto non cambia**. Le etichette sono le frasi di evidenza annotate nei dataset (SciFact, WiCE, HoVer), riportate sulle frasi del nostro divisore. Siccome il modello è congelato, i vettori delle frasi si calcolano **una volta** e si salvano su disco: poi ogni prova di training della testa dura minuti.

2.6 Il divisore in frasi fa parte del modello

La testa lavora su frasi: in produzione il documento va diviso **esattamente** come in training, con la stessa funzione e la stessa versione dei suoi dati (NLTK `punkt_tab`). Se le frasi fossero tagliate in modo diverso, la testa vedrebbe unità diverse da quelle su cui è stata addestrata. Per questo il divisore è versionato con il modello (hash nel manifest e in `decision.json`), e conviene includere nella repo i parametri di `punkt_tab` (passo 12 §4).

3. I modelli

I 10 decision model dello studio focalizzato, tutti costruiti su **Qwen3.5**: 32 layer, di cui 8 ad attenzione piena (layer 3, 7, 11, 15, 19, 23, 27, 31) e 24 Gated DeltaNet (attenzione lineare, sempre causale).

Modello	Ruolo	d	Attenzione nei layer pieni
KnowLine-4B (`PelaAI/KnowLine-4B-Gen5.5`)	tutte le prove si fanno qui	2.560	causale
elephant-1 (`baselabs/elephant-1`)	conferma a 9B	4.096	bidirezionale a blocchi
Jeffan-4B, d3-flash, OneJev1.6-9B, Nimble-9B-v3	estensione	2.560 / 4.096	Jeffan, d3: bidirezionale; OneJev, Nimble: causale
StartLux 9B/4B, pplx-decider-flash, Clef-Flash	estensione, più lavoro	2.560 / 4.096	StartLux, pplx: bidirezionale; Clef: come caricato

Ogni modello si esegue **con il codice dei suoi autori** (prompt, maschera di attenzione, lettura della risposta), come nei benchmark della repo. Dettagli nel passo 01.

4. Glossario

Termine	Significato
forward	un passaggio del modello su tutto il prompt
stato nascosto, vettore	i numeri che il modello calcola per un token in un layer
posizione di risposta	il token del prompt dove si legge il verdetto (l'ultimo)
testa di evidenza	la piccola rete che dà un punteggio a ogni frase
`s[k]`, `s0`	punteggio della frase k; soglia del documento
frase gold	frase di evidenza annotata nel dataset, riportata sulla nostra divisione
insieme completo	tutte le frasi gold mostrate (per l'evidenza sparsa conta questo)
sufficienza	le frasi mostrate, da sole come documento, fanno ancora dire "supportato" al modello
fase 1a / 1b / 2	training della testa su vettori salvati / con il modello in memoria / insieme a una LoRA del modello

5. Sequenza di lavoro

#	Passo	File	Prodotto
1	Modelli e hook per frase	01	forward con cattura dei vettori e dell'attenzione per frase
2	Dati dalle fonti	02	`dataset/evidence/*.jsonl` (già costruiti)
3	Dati generati (facoltativo per le prime prove)	03	evidenza nota per costruzione, etichette dal modello, evidenza spostata
4	Split e mix	04	set congelati, mix di training
5	Prompt con il preambolo	05	preambolo scelto, verdetto verificato
6	Dalle frasi ai token, estrazione	06	vettori delle frasi su disco
7	Baseline	07	numeri da battere
8	Architettura e loss	08	`evidence/head.py`, `evidence/losses.py`
9	Training	09	teste addestrate
10	Valutazione	10	metriche e latenza
11	Esperimenti	11	configurazione finale
12	Server e rilascio	12	server che restituisce verdetto ed evidenza
—	Checklist	13	
—	Una strada da esplorare	14	
—	Riepilogo: architettura, layer, loss	15	

Codice già pronto nella repo: `evidence/sentences.py` (divisore), `evidence/data/` (costruzione dei dataset del passo 2), `tests/test_evidence_data.py`. Il resto è da scrivere, con i file indicati in ogni passo.

6. Obiettivi

Obiettivo	Soglia
Verdetto	uguale a quello del fine-tuning binario (BAcc e AUC entro 0,3 punti)
F1 sulle frasi	almeno 15 punti sopra la baseline lessicale (E-LEX, 07); indicativo ≥ 0,75 su HoVer
Insieme completo su HoVer	≥ 0,65
Evidenza lontana	F1 con frasi gold a ≥ 20 frasi di distanza al massimo 5 punti sotto quello con frasi vicine
Sufficienza	≥ 85%
Latenza	al massimo +3% del solo verdetto (modelli 4B: +3% o +2 ms)

Passo 1 — I modelli e gli hook per frase

In breve

Serve un modulo, `evidence/models.py`, che per ciascun modello faccia **un forward** con il codice dei suoi autori e restituisca, dallo stesso forward:

1. i punteggi delle due opzioni nella posizione di risposta → P(supportato), come nel fine-tuning binario;
2. per ogni frase del documento, in **4 layer**: il vettore dell'ultimo token della frase (`h_end`) e la media dei vettori dei suoi token (`h_mean`);
3. per gli 8 layer ad attenzione piena, quanto la posizione di risposta "guarda" ogni frase (somma dei pesi di attenzione sui suoi token, per testa);
4. il vettore della posizione di risposta (`q`), negli stessi 4 layer.

I punti 2–4 si ottengono con degli *hook*: piccole funzioni agganciate ai layer, che copiano i dati mentre il forward passa, senza cambiarlo. Il riassunto per frase si calcola subito sulla GPU: non si conservano mai i vettori di tutti i token.

1. Pacchetti

Ambiente `factcheck` della repo (transformers 5.14, torch, accelerate, peft), più:

```
pip install flash-linear-attention causal-conv1d safetensors nltk rank_bm25 sentence-transformers scikit-learn
python -c "import nltk; nltk.download('punkt_tab')"      # dati del divisore in frasi
chmod -R go-w ~/nltk_data                                # NLTK rifiuta una cartella dati scrivibile da altri utenti
```

`flash-linear-attention` e `causal-conv1d` servono ai 24 layer Gated DeltaNet di Qwen3.5: senza, transformers usa un'implementazione lenta in torch.

2. L'architettura comune: Qwen3.5

	Gruppo 4B	Gruppo 9B
Layer	32	32
Layer ad attenzione piena	3, 7, 11, 15, 19, 23, 27, 31	idem
Altri layer	24 Gated DeltaNet: portano avanti un riassunto del testo letto, sempre causali	idem
Teste di query / di chiavi (layer pieni)	16 / 4	16 / 4
Lunghezza del vettore (d)	2.560	4.096
Tokenizer	Qwen3.5 (comune a tutti)	idem

Layer da cui si leggono i vettori delle frasi: **19, 23, 27, 31**, gli ultimi quattro ad attenzione piena (il 31 è anche l'ultimo layer). Perché questi e come verificarli con una sonda: passo 15 §3.

3. Come si esegue ciascun modello

Ogni modello si esegue con il codice dei suoi autori: prompt, maschera di attenzione e lettura della risposta sono i loro. Caricare i pesi con transformers "normale" darebbe, per i modelli bidirezionali, un modello causale che risponde diversamente **senza dare errori**.

Modello	Codice degli autori	Prompt (schema)	Attenzione	Lettura della risposta	Posizione di risposta
KnowLine-4B	front end nel repository del modello (`knowline_server.py`, `knowline_layout.py`), backend transformers	stato come testo indentato in `key: value`, un turno per riga; turno finale con istruzione, `Question:`, `A. Yes: …` / `B. No: …`; risposta aperta con `Answer:`. Richieste fino a 3.000 token **scritte due volte**	causale	token A/B; temperatura 1,50 già nei pesi	ultimo token
elephant-1	motore `qev` di OneJev (copia in `models/elephant-1/qev`)	sistema + `<state>` JSON indentato `</state>` come prefisso; la domanda come suffisso con `A. yes` / `B. no`	bidirezionale a blocchi nei layer pieni (prefisso con sé stesso, suffisso con prefisso e sé stesso), dichiarata in `modeling_overrides` del config	lettere A/B	ultimo token del suffisso
Jeffan-4B	`inference/jeffan_engine` (nel repository)	`State:` JSON, `Question:`, `A: No / false`, `B: Yes / true`	bidirezionale nei layer pieni	codici A/B	ultimo token
d3-flash	codice del modello (`trust_remote_code`)	`State:` JSON, `Question:`, `A: No / false`, `B: Yes / true`	non causale nei layer pieni	codici fra 255	ultimo token
OneJev1.6-9B	renderer di Flide; in transformers come sequenza unica	prefisso (sistema + stato) e suffisso della domanda	causale	lettere A/B	ultimo token
Nimble-9B-v3	motore degli autori (`models/nimble-9b/nimble`), LoRA su Qwen3.5-9B	turno JSON con contesto e schema, `Requested field: "<id>"`	causale	codici fra 255	ultimo token
StartLux 9B/4B	pacchetto `startlux-decide` (gli autori usano vLLM con un plugin; in transformers la maschera va ricostruita)	`Evidence:` JSON compatto, `Question:`, `A) yes` / `B) no`	bidirezionale sul prompt	lettere A/B, temperatura per tipo	ultimo token

Modello	Codice degli autori	Prompt (schema)	Attenzione	Lettura della risposta	Posizione di risposta
pplx-decider-flash	pacchetto `pplx_decider` (percorso di riferimento)	domande in una sequenza, ognuna letta nella sua posizione	non causale nei layer pieni	testa propria a 255 vie	posizione di lettura della domanda
Clef-Flash	`joint_schema_model.py`	`STATE:` JSON, `SCHEMA FIELDS:`	come caricato	testa congiunta propria	ultimo token prima della testa

4. Trovare i layer

```python
import torch

def text_layers(model, n_layers=32):
    """La ModuleList dei layer testuali (i modelli caricati come multimodali hanno anche una torre visiva)."""
    found = [m for name, m in model.named_modules()
             if isinstance(m, torch.nn.ModuleList) and len(m) == n_layers and "visual" not in name and "vision"
              ↪  not in name]
    assert len(found) == 1
    return found[0]

def full_attention_layers(config):
    tc = getattr(config, "text_config", config)
    return [i for i, t in enumerate(tc.layer_types) if t == "full_attention"]     # [3, 7, 11, 15, 19, 23, 27, 31]
```

5. Hook sui vettori delle frasi

Per ognuno dei 4 layer, un hook prende l'uscita del layer (un vettore per token) e calcola subito, sulla GPU:

- `q`: il vettore della posizione di risposta;
- `end`: il vettore dell'ultimo token di ogni frase;
- `mean`: la media dei vettori dei token di ogni frase, con le somme cumulative (un solo passaggio per tutte le frasi).

Gli intervalli di token delle frasi (`spans`) vengono dal rendering (06 §3).

```python
import torch


class SentenceTap:
    """Per ogni layer scelto: vettore della posizione di risposta, dell'ultimo token di ogni frase, media dei
     ↪  token di ogni frase."""
    def __init__(self, layers, layer_idx):
        self.out, self.spans, self.answer_pos = {}, None, None       # spans: LongTensor [n, 2], token del
         ↪  prompt, estremi inclusi
        self.handles = [layers[i].register_forward_hook(self._hook(i)) for i in layer_idx]

    def _hook(self, i):
        def hook(module, args, output):
            h = (output[0] if isinstance(output, tuple) else output)[0]          # [T, D]
            s = self.spans
            csum = torch.cat([h.new_zeros(1, h.size(1), dtype=torch.float), h.float().cumsum(0)])   # somme
↪  cumulative
            mean = (csum[s[:, 1] + 1] - csum[s[:, 0]]) / (s[:, 1] - s[:, 0] + 1)[:, None]
            self.out[i] = {"q": h[self.answer_pos].float(), "end": h[s[:, 1]], "mean": mean.to(h.dtype)}
        return hook


def attention_per_sentence(rows, spans):
    """rows: {layer: [16, T]} righe di attenzione della posizione di risposta; spans: LongTensor [n, 2].
    -> [n, 8 × 16]: per ogni frase, la somma dei pesi sui suoi token, per layer e testa."""
    feats = []
    for layer in sorted(rows):
```

```
        csum = torch.cat([rows[layer].new_zeros(rows[layer].size(0), 1), rows[layer].float().cumsum(1)], 1)   #
↪  [16, T+1]
        feats.append((csum[:, spans[:, 1] + 1] - csum[:, spans[:, 0]]).T)                                       # [n,
↪  16]
    return torch.cat(feats, 1)
```

Provato con un modello finto: la media e l'ultimo token coincidono con il calcolo diretto, e la somma dell'attenzione per frase con la somma sui token della frase, per ogni layer e testa.

6. Hook sull'attenzione

Le implementazioni veloci dell'attenzione non restituiscono i pesi, e chiederli in modalità "eager" costa troppa memoria con documenti lunghi. Si registra una funzione di attenzione che calcola l'attenzione normale e, negli 8 layer pieni, ricalcola **una riga** di pesi: quella della posizione di risposta verso tutti i token. Costa pochissimo.

```
from transformers import AttentionInterface
from transformers.integrations.sdpa_attention import sdpa_attention_forward

CAPTURE = {"rows": {}, "full": {3, 7, 11, 15, 19, 23, 27, 31}, "query_pos": -1, "inner": sdpa_attention_forward}

def sdpa_capture(module, query, key, value, attention_mask, scaling=None, **kw):
    out = CAPTURE["inner"](module, query, key, value, attention_mask, scaling=scaling, **kw)
    if module.layer_idx in CAPTURE["full"]:
        qp = CAPTURE["query_pos"]
        sl = slice(qp, qp + 1 if qp != -1 else None)
        k = key.repeat_interleave(query.size(1) // key.size(1), dim=1)          # 16 teste di query, 4 di chiavi
        s = (query[:, :, sl] @ k.transpose(-1, -2)).float() * scaling            # [B, 16, 1, T]
        if attention_mask is not None:                                          # la maschera del modello, anche
        ↪  bidirezionale
            s = s + attention_mask[:, :, sl, : s.size(-1)].float()
        CAPTURE["rows"][module.layer_idx] = s.softmax(-1)[:, :, 0]               # [B, 16, T]
    return out

AttentionInterface.register("sdpa_capture", sdpa_capture)
# caricamento: from_pretrained(..., attn_implementation="sdpa_capture")
```

- In transformers 5.14 la classe di attenzione di Qwen3.5 passa per `AttentionInterface` e ha `layer_idx`; `query` e `key` arrivano già pronte (dopo le rotazioni di posizione), `scaling` arriva dal modello.
- Se il codice degli autori installa una propria funzione di attenzione per la maschera bidirezionale (pplx, Jeffan, d3-flash, elephant-1), la si mette in `CAPTURE["inner"]` al posto di `sdpa_attention_forward`: la maschera resta la loro.

Dalle righe di pesi alla feature per frase: `attention_per_sentence` (§5) somma i pesi sui token di ogni frase, per ognuno degli 8 layer e delle 16 teste: 128 numeri per frase.

7. Che cosa restituisce `evidence/models.py`

```
def forward_sentences(model, rendered):
    """rendered: input_ids, answer_pos, sent_tok_spans (primo e ultimo token di ogni frase nel prompt) -> dict:
       answer_logits [2]          punteggi delle opzioni sì/no (per P(supportato))
       q      [4, D]              vettore della posizione di risposta, layer 19, 23, 27, 31
       h_end  [n, 4, D]           vettore dell'ultimo token di ogni frase
       h_mean [n, 4, D]           media dei token di ogni frase
       att    [n, 128]            attenzione della posizione di risposta sulle frasi
       n_tok  [n]                 numero di token di ogni frase"""
```

8. Test

`python -m evidence.models --model knowline-4b --selftest`, poi per gli altri modelli:

Test	Come	Deve dare
Verdetto invariato	50 righe di dev, prompt senza preambolo: P(supportato) dal forward con gli hook contro l'adapter del benchmark	differenza media < 0,005, massima < 0,02
Layer giusti	vettore dell'hook al layer i = `hidden_states[i + 1]` di transformers	identico
Attenzione giusta	5 prompt da ~1.000 token, contro la modalità "eager" con la stessa maschera	differenza < 0,001
Maschera conservata	per i modelli bidirezionali, il verdetto con e senza hook	identico
Riassunti delle frasi	`end` e `mean` contro il calcolo diretto sugli stati completi	differenza < 1e-3
Memoria	forward da 8.000 e 16.000 token con gli hook	entra nella GPU (§9)

9. GPU e spazio

Uso	Gruppo 4B	Gruppo 9B
Estrazione dei vettori	24 GB	45 GB per documenti lunghi
Training con il modello in memoria (fase 1b)	24 GB	45 GB
Training anche del modello (fase 2)	24 GB	45 GB

Spazio su disco, 4 layer, bf16: per frase 2 × 4 × D × 2 byte (82 KB su KnowLine-4B, 131 KB su elephant-1). Per le ~28.600 righe di SciFact, WiCE e HoVer (~52 frasi in media) circa 120 GB su KnowLine-4B; con 2 layer la metà.

Passo 2 — I dati dalle fonti

In breve

Per addestrare e valutare la testa servono righe con documento, claim, etichetta, il documento **diviso in frasi** e, per i positivi, **quali frasi sono l'evidenza**. Si usano tre dataset pubblici che hanno già l'evidenza annotata: **SciFact**, **WiCE**, **HoVer** (con HotpotQA distractor per il testo delle pagine).

La costruzione è già scritta e funzionante, in `evidence/data/` (comando `python -m evidence.data.build`), e produce esattamente il formato che serve alla testa per frase: niente da aggiungere.

1. Le fonti

Fonte	Da dove	Righe	Positivi / negativi	Documento
SciFact	`allenai/scifact_entailment` (cache HF), train + validation	1.258	508 / 750	titolo + abstract biomedico (mediana 9 frasi)

Fonte	Da dove	Righe	Positivi / negativi	Documento
WiCE	`tasksource/wice` (cache HF), train + dev + test	5.294	3.100 / 2.194	la pagina web citata da Wikipedia, una riga per elemento (mediana 68 frasi)
HoVer	claim da `dataset/HoVer/data/hover_{train,dev}_release_v1.1.json`; testo da `hotpotqa/hotpot_qa` (distractor)	22.063	12.974 / 9.089	~10 pagine Wikipedia: quelle dell'evidenza più pagine di disturbo (mediana 46 frasi)

SciFact

- `SUPPORT` → positivo; le frasi di *rationale* sono l'evidenza.
- `CONTRADICT` → negativo; le frasi che contraddicono si salvano a parte (`contra`), solo per analisi.
- `NEI` → negativo.
- Documento: titolo, a capo, frasi dell'abstract unite da spazio.

WiCE

- `supported` → positivo; `supporting_sentences` dà **più insiemi alternativi** di frasi che dimostrano il claim.
- `not_supported` → negativo.
- `partially_supported` → etichetta 0 ma **nessuna loss di evidenza** (una parte del claim è supportata, quindi non si può chiedere che nessuna frase si accenda), oppure scartato con `--wice-partial drop`.
- Documento: gli elementi della pagina (titoli, righe, frasi) uniti da a capo.
- **WiCE è anche una fonte del test LLM-AggreFact.** Ogni riga ha `meta.aggrefact_overlap`: `"dev"` o `"test"` se la stessa pagina è in AggreFact dev o test, `false` altrimenti. Il confronto si fa sulla pagina, non sul claim, perché AggreFact usa i claim interi e WiCE li divide in pezzi. Misurato: tutto il dev di WiCE è in AggreFact dev, tutto il test di WiCE in AggreFact test, più 2 righe del train. Queste righe non vanno mai in training (04).

HoVer con HotpotQA

- HoVer dà i claim (2–4 "salti" fra pagine), le etichette `SUPPORTED` / `NOT_SUPPORTED` e le frasi di evidenza come (titolo della pagina, numero di frase), con la numerazione di HotpotQA.
- HotpotQA distractor, collegato con `hpqa_id`, dà per quella domanda le pagine già divise in frasi e ~8 pagine di disturbo. Il documento contiene le pagine dell'evidenza e quelle di disturbo, in ordine casuale fissato dall'id del claim: ogni pagina è titolo, a capo, testo; pagine separate da una riga vuota. L'evidenza è spesso lontana: in 6.608 positivi su 12.974 le frasi gold distano 20 o più frasi.
- Per i claim a 3–4 salti, HotpotQA non ha tutte le pagine. Con `--hover-db-fallback` le pagine mancanti si prendono dal database Wikipedia di HoVer (`dataset/HoVer/data/wiki_wo_links.db`), e la frase n si trova dividendo il testo con il nostro divisore. Senza questa opzione si perde il 45% dei claim. Misurato su 5.488 frasi di pagine presenti in entrambe le fonti: la frase così trovata è quella giusta nel 98% dei casi; nel 2% è sfasata di una frase. Queste righe hanno `meta.gold_alignment = "db_split"`, le altre `"exact"`.

2. Il divisore in frasi

`evidence/sentences.py`, funzione `split(testo)`, restituisce gli intervalli `[inizio, fine)` di caratteri delle frasi. È **l'unica** divisione usata in tutta la pipeline: dati, rendering, valutazione e server. Le regole:

1. si va sempre a capo dopo una riga vuota, prima di un elemento di elenco (`-`, `*`, `1.`) e dopo una riga che non finisce con punteggiatura (un titolo); una riga che finisce con virgola o trattino continua nella successiva;
2. dentro ogni blocco si usa NLTK Punkt (modello `punkt_tab`, inglese), con alcune abbreviazioni in più che il modello non conosce: vol, vols, no, fig, figs, eq, eqs, approx, ca, cf, mt, sgt, capt, gov, dept, pp, al, e.g, i.e (così "Vol. 2 No. 25" non viene spezzato);
3. frasi oltre 60 parole si spezzano su `;` o `:` vicino a metà, poi su `,`, poi su uno spazio;
4. frasi sotto 3 parole si uniscono alla successiva dello stesso paragrafo (alla precedente se sono le ultime);
5. oltre 512 frasi la riga si esclude.

Le frasi di evidenza delle fonti si riportano sulla nostra divisione: una nostra frase è evidenza se almeno metà dei suoi caratteri, o metà della frase della fonte, cade in una frase di evidenza della fonte. Così training e produzione vedono gli stessi confini. Misurato: la frase della fonte corrisponde a una sola nostra frase nel 96,5% dei casi (SciFact), 97,3% (WiCE), 98,3% (HoVer).

Se si cambia `sentences.py`, i dataset vanno ricostruiti e le teste riaddestrate: il suo sha256 sta nel manifest.

3. Preparazione e comando

```
python -c "import nltk; nltk.download('punkt_tab')"
chmod -R go-w ~/nltk_data
hf download hotpotqa/hotpot_qa --repo-type dataset --include "distractor/*"
python -m evidence.data.build --source scifact wice hover --hover-db-fallback
```

SciFact e WiCE devono essere nella cache HF; HoVer in `dataset/HoVer/`; LLM-AggreFact in `dataset/lytang--LLM-AggreFact/` (per il controllo su WiCE); il tokenizer di KnowLine-4B nella cache HF (per il filtro di lunghezza).

Opzione	Default	Cosa fa
`--hover-db-fallback`	spento	pagine HoVer mancanti prese dal db (da tenere acceso)
`--max-tokens N`	32.768	scarta le righe il cui prompt supera N token del tokenizer di KnowLine-4B: documento + 2 × claim + 256 (il front end di KnowLine aggiunge 157 token, misurati)
`--wice-partial`	`negative`	WiCE parzialmente supportati: etichetta 0 senza loss di evidenza, o `drop`
`--limit N`	0	primi N record per fonte, per prove

Uscita in `dataset/evidence/`: `<fonte>.jsonl`, `<fonte>.stats.json` (conteggi, scarti con motivo, lunghezze, distanze fra frasi gold, allineamento), `manifest.json` (file letti, revisioni, sha256, opzioni, commit, sha256 di `sentences.py`). Circa un minuto e mezzo.

Con il limite di default non si scarta nessuna riga per lunghezza (il prompt più lungo è di 18.580 token, in WiCE). Con `--max-tokens 8192` si scartano 75 righe di WiCE.

4. Il formato di una riga


```
{"id": "hover:train:330ca632-…", "source": "hover", "orig_split": "train",
 "claim": "…", "document": "Kristian Zahrtmann\nKristian Zahrtmann was …\n\nOssian Elgström\n…",
 "label": 1,
 "sentences": [[0, 61], [62, 140], "…"],
 "evidence": [1, 9, 31], "evidence_sets": [[1, 9, 31]],
 "contra": [],
 "evidence_quality": "gold", "evidence_term": "supervised",
 "doc_id": "hover:5ab7a86d5542995dae37e986",
 "meta": {"orig_label": "SUPPORTED", "gold_alignment": "exact", "doc_tokens": 1534, "prompt_tokens": 1830, "…":
  ↪  "…"}}
```


Campo	Significato
`label`	1 = supportato, 0 = no
`sentences`	inizio e fine di ogni frase in caratteri (`document[s:e]`), dal nostro divisore
`evidence`	indici delle frasi di evidenza; vuoto se `label = 0`
`evidence_sets`	gruppi alternativi di frasi che dimostrano il claim (WiCE ne ha più di uno); `evidence` è la loro unione
`contra`	frasi che contraddicono il claim (SciFact), solo per analisi
`evidence_term`	che loss riceve la riga (08 §6): `supervised` (positivo con evidenza), `mil_neg` (negativo: nessuna frase deve superare la soglia), `none` (nessuna loss di evidenza)
`doc_id`	righe con lo stesso documento (lo split si fa per `doc_id`)
`meta.gold_alignment`	`exact` o `db_split` (HoVer)
`meta.prompt_tokens`	lunghezza stimata del prompt in token

I file si leggono anche con il benchmark della repo, perché hanno `document`, `claim`, `label`, `source`, `id`: `python benchmark.py --dataset evidence --split scifact --model knowline-4b --gpu 0`.

5. Statistiche dei dataset costruiti

	SciFact	WiCE	HoVer
Righe	1.258	5.294	22.063
Documenti diversi	667	1.933	7.619
Frasi per documento (mediana)	9	68	46
Frasi di evidenza per positivo (media)	1,8	2,6	3,0
Positivi con ≥ 2 frasi di evidenza	51%	70%	100%
Positivi con frasi gold a ≥ 20 frasi di distanza	3	583	6.608
Token del prompt, mediana / massimo	590 / 1.739	1.668 / 18.580	1.690 / 4.099

6. Controlli

Controllo	Atteso
Le frasi coprono tutto il documento senza sovrapporsi	tutte le righe (lo verifica il build)
Frasi della fonte corrispondenti a una sola nostra frase	≥ 90% per fonte (il build si ferma se sotto)
30 positivi per fonte letti a mano (`python -m evidence.data.show --source hover --n 30`)	le frasi marcate dimostrano il claim in almeno 28 su 30

Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata

In breve

Le tre fonti del passo 2 hanno due limiti: i testi sono quasi solo Wikipedia, pagine web e abstract biomedici, e solo HoVer ha molta evidenza lontana. Tre famiglie di dati in più li correggono. Per le prime prove si possono saltare; servono quando la testa funziona e si vuole migliorarla.

Famiglia	Idea	Che cosa porta	Script
Generati con posizioni note	un LLM scrive un articolo che contiene fatti dati; si sa dove sono	evidenza su testi tipo notizia, anche multi-frase	`evidence/data/gen_marked.py`
Etichettati col modello	per documenti senza evidenza annotata, si cerca col modello il gruppo più piccolo di frasi che basta a fargli dire “supportato”	evidenza su testi senza annotazioni	`evidence/data/label_ablation.p`
Evidenza spostata	si allontanano le frasi gold, o si inseriscono frasi di disturbo	più casi di evidenza lontana	`evidence/data/augment_sparse.p`

Tutte scrivono righe nel formato del passo 2 (frasi del nostro divisore, indici di evidenza), validate dallo stesso `evidence/data/schema.py`. Si generano **solo da righe di training**.

1. Documenti generati con posizioni note

Si parte da claim scomposti in fatti semplici (ricetta C2D di MiniCheck: ogni claim è riscritto come 2–4 frasi atomiche che insieme lo implicano). Al generatore (un LLM aperto in locale) si chiede:

```
Write a coherent news-style article of 15 to 40 sentences that includes each of the
following sentences word for word, each wrapped in its tag: <e1>…</e1>, <e2>…</e2>, …
Add other context sentences without tags. Do not restate the information of the
tagged sentences anywhere else in the article. Place the tagged sentences far apart.

<e1>{fatto 1}</e1>
<e2>{fatto 2}</e2>
```

Poi:

1. ogni `<ei>…</ei>` deve comparire una volta sola e parola per parola, altrimenti la riga si scarta;
2. si tolgono i tag, registrando l’intervallo di caratteri di ogni testo marcato (con `DocBuilder` di `evidence/data/schema.py`);
3. si divide il testo con `evidence/sentences.py`; le frasi di evidenza sono quelle che coprono gli intervalli marcati (stessa regola di allineamento del passo 2); se un testo marcato finisce in più di 2 frasi, la riga si scarta;

4. un giudice LLM diverso conferma: “do these sentences, taken alone, establish the claim? yes/no”; si tengono solo i “yes”.

Varianti dallo stesso articolo:

- **negativo**: si toglie la frase di un fatto necessario → il claim non è più supportato, `evidence_term = mil_neg`;
- **claim parziale**: il claim si riscrive usando solo alcuni fatti → supportato, con evidenza = solo le frasi di quei fatti. È un esempio utile: l’evidenza è un sottoinsieme preciso del documento.

`evidence_quality = construct`, peso della riga 1.

2. Etichette ricavate col modello (ablazione)

Per righe supportate senza evidenza annotata (per esempio i dati C2D e D2C di MiniCheck come rilasciati). Si usa il verdetto di KnowLine-4B con il prompt con preambolo, e si cerca il gruppo più piccolo di frasi che, da solo, fa dire “supportato”:

```
def ablation_label(n_sentences, p_supported, thr, max_k=6):
    """p_supported(indici) = P(supportato) con il documento ridotto a quelle frasi (nell'ordine originale).
    -> (indici delle frasi di evidenza, qualità) oppure None."""
    chosen = []
    while len(chosen) < max_k:
        best = max((k for k in range(n_sentences) if k not in chosen), key=lambda k: p_supported(sorted(chosen +
↪   [k])))
        chosen.append(best)
        if p_supported(sorted(chosen)) > thr:
            break
    else:
        return None                                  # nessun gruppo di al massimo 6 frasi basta: riga scartata
    for k in list(chosen):                               # pulizia: via le frasi che non servono
        rest = [j for j in chosen if j != k]
        if rest and p_supported(sorted(rest)) > thr:
            chosen = rest
    complement = [k for k in range(n_sentences) if k not in chosen]
    redundant = p_supported(complement) > thr            # l'evidenza è ripetuta altrove nel documento
    return sorted(chosen), ("ablation_partial" if redundant else "ablation")
```

- Ogni passo greedy manda tutti i documenti ridotti in un batch; KnowLine è causale, quindi si riusa il calcolo della parte iniziale comune del prompt.
- `thr` = soglia del verdetto.
- Filtri: un giudice LLM conferma le frasi trovate (atteso almeno l’80% di conferme); su 300 righe si rifà la ricerca con elephant-1, e i gruppi devono coincidere in buona parte (Jaccard medio ≥ 0,7).
- `evidence_quality = ablation`, peso della riga 0,5 (`ablation_partial`: 0,3).

L’ablazione rende l’evidenza **fedele al modello** che etichetta: la testa di un altro modello addestrata su queste etichette impara ciò che basta a KnowLine. Il controllo con elephant-1 serve a questo.

3. Evidenza spostata

Da righe con almeno 2 frasi gold:

Trasformazione	Come
Agli estremi	la prima frase gold va all’inizio del documento, l’ultima alla fine
Frasi di disturbo	5–40 frasi di altri documenti di training della stessa fonte inserite fra le frasi gold
Lontano dall’inizio	testo di disturbo aggiunto in testa finché la prima frase gold è oltre 1.024 token (o oltre 4.096)
Negativi spostati	le stesse operazioni su righe non supportate

Il documento nuovo si ricostruisce pezzo per pezzo con `DocBuilder`; poi si ridivide con `sentences.py` e gli indici di evidenza si ricalcolano con la regola di allineamento.

Controlli:

- una frase gold che inizia riferendosi alla precedente (“He”, “It”, “This”, “The company”…) non si sposta da sola: si sposta con la precedente, o la riga si scarta;
- il giudice conferma che l’etichetta non è cambiata;
- le frasi di disturbo vengono solo da documenti di training.

4. Quante righe

Famiglia	Prima prova	Versione completa
Generati con posizioni note	2.000	10.000
Etichettati col modello	2.000	10.000
Evidenza spostata (positivi)	4.000	10.000
Negativi spostati	1.000	3.000

5. Controlli

Controllo	Atteso
30 righe per famiglia lette a mano: le frasi di evidenza dimostrano il claim e non ne manca nessuna necessaria	almeno 27 su 30
Conferme del giudice sull’ablazione	≥ 80%
Accordo KnowLine–elephant-1 sull’ablazione	Jaccard ≥ 0,7
Righe spostate con riferimenti rotti (30 lette)	0

Passo 4 — Split e mix di training

In breve

1. **Split**: si decide una volta quali righe servono per addestrare (train), per scegliere (dev) e per la misura finale (test), e non si cambia più. Si divide per documento (`doc_id`): lo stesso documento non può stare in due split.
2. **Mix** `ev_head`: le righe di training con il loro tipo di loss e i loro pesi.

Script: `evidence/data/make_splits.py`, `evidence/data/make_mix.py`. Uscita in `dataset/evidence/splits/` e `dataset/evidence/mixes/`, con un `manifest.json` (hash dei file, conteggi, seme 13).

1. Regole degli split

Fonte	Train	Dev	Test
SciFact	90% del train d’origine (per `doc_id`)	10% del train d’origine	validation d’origine
HoVer	95% del train d’origine (per `doc_id`)	5% del train d’origine, solo righe `exact`	dev d’origine, solo righe `exact`
WiCE	train d’origine **senza** le righe con `aggrefact_overlap`	—	dev + test d’origine (tutte con `aggrefact_overlap`): test fuori dominio

Perché:

- **WiCE**: le pagine del suo dev e test sono le stesse del dev e test di LLM-AggreFact, il benchmark finale; addestrare su di esse falserebbe quella misura. Restano utili come test fuori dominio (pagine web, non Wikipedia).
- **HoVer `db_split`** (pagine prese dal db, frase sfasata nel 2% dei casi): si usano in training con peso minore, ma non per misurare.
- I `doc_id` di dev e test non compaiono mai nel train; i dati generati e spostati nascono solo dal train.

2. I set

Set	Contenuto	Uso
`train`	il resto	training della testa
`dev_evid_select`	metà dei positivi di dev (SciFact, HoVer), divisa per `doc_id`	tutte le scelte: configurazione, soglia di selezione, esperimenti
`dev_evid_confirm`	l'altra metà	conferma delle scelte
`dev_evid_neg`	negativi del dev con documenti lunghi	controllare che la testa non dia punteggi sopra soglia su claim non supportati
`test_evid_id`	test di HoVer (`exact`) e SciFact	misura finale
`test_evid_ood`	WiCE dev + test	misura finale fuori dominio
`test_evid_sparse`	da `test_evid_id`, con l'evidenza spostata lontano (passo 3, solo con frasi di disturbo del test)	evidenza lontana

Per ogni set si salvano: frasi gold per riga, distanza fra la prima e l'ultima frase gold (in frasi: vicine / 2–9 / 10–19 / ≥ 20; e in token dall'inizio del documento: < 1.024 / 1.024–4.096 / > 4.096), lunghezza in token.

3. Il mix `ev_head`

Ogni riga riceve il tipo di loss dal suo `evidence_term` (08 §6) e due pesi:

- **peso della riga** (`row_weight`): quanto conta la riga nella media del batch;
- **ω** (`omega`): quanto pesano le frasi **non** gold. Basso dove le annotazioni sono incomplete (frasi valide non segnate dagli annotatori).

Famiglia	Termine	Peso della riga	ω
HoVer supportati, `exact`	`supervised`	1,0	0,5
HoVer supportati, `db_split`	`supervised`	0,5	0,5
SciFact SUPPORT	`supervised`	1,0	0,5
WiCE supportati (train)	`supervised`	1,0	0,3
Generati con posizioni note (passo 3)	`supervised`	1,0	1,0
Etichettati col modello (passo 3)	`supervised`	0,5	0,5
Negativi: HoVer NOT_SUPPORTED, SciFact CONTRADICT/NEI, WiCE not_supported	`mil_neg`	1,0	—

Famiglia	Termine	Peso della riga	ω
WiCE parzialmente supportati	`none`: fuori dal mix della testa	—	—

ω è più basso per WiCE perché i suoi gruppi di evidenza sono spesso incompleti (altre righe della pagina possono supportare il claim).

Con le sole tre fonti il mix ha circa 14.000 positivi e 10.000 negativi (HoVer ne è la gran parte).

Vincoli che lo script controlla e stampa:

Vincolo	Valore
Negativi	20–45% delle righe (con le tre fonti sono ~40%: si può sottocampionare a 30%)
Positivi con 2 o più frasi gold	≥ 40%
Positivi con frasi gold a 10 o più frasi di distanza	≥ 25%
Positivi con evidenza oltre 1.024 token dall'inizio del documento	≥ 15%
Documenti in comune con dev o test	0
Righe WiCE con `aggrefact_overlap`	0

4. Comandi

```
python -m evidence.data.make_splits --out dataset/evidence/splits
python -m evidence.data.make_mix --recipe recipes/ev_head.json --out dataset/evidence/mixes/ev_head.jsonl
```

La ricetta (`recipes/ev_head.json`) contiene famiglie, numeri, pesi e seme: lo stesso mix si ricostruisce identico.

Passo 5 — Il prompt con il preambolo

In breve

Si aggiunge una riga con il claim **prima** dello stato (documento e claim), lasciando il resto del prompt identico a quello del fine-tuning binario. Poi si controlla che il verdetto funzioni ancora con questa riga in più: se sì, si riusa così com'è; se no, si riaddestra con la stessa ricetta del fine-tuning binario, includendo la riga.

1. Perché serve

Nei modelli causali (KnowLine-4B, OneJev, Nimble) ogni token vede solo i token che lo precedono. Nel prompt del fine-tuning binario lo stato è "documento, poi claim": quando il modello legge il documento non sa ancora che cosa deve verificare, e i vettori delle frasi (quelli che la testa usa) non dipendono dal claim. Con il claim scritto anche prima, ogni frase del documento è letta "sapendo cosa si cerca".

Nei modelli bidirezionali (elephant-1, StartLux, d3-flash, pplx, Jeffan) le frasi del documento vedono il claim negli 8 layer ad attenzione piena anche senza preambolo, ma non nei 24 layer DeltaNet, che restano causali. Il preambolo serve meno; l'esperimento H8 (11) misura quanto.

Il claim resta anche nella posizione originale, dopo il documento: lì il verdetto è stato addestrato a vederlo.

2. Esempio (KnowLine-4B, schema)

Il testo esatto lo produce il front end degli autori (`knowline_layout.py`). Schema:

```
senza preambolo
[turno utente]       document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda> Options: A. Yes: … B. No: …
[turno assistente]   Answer:

con preambolo
[turno utente]       Claim under review: The meeting is on Tuesday.

                     document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda> Options: A. Yes: … B. No: …
[turno assistente]   Answer:
```

KnowLine scrive **due volte** le richieste fino a 3.000 token (“Read the same input again before answering:”). Il preambolo fa parte dello stato e compare in entrambe le copie. Le frasi per la testa si leggono dalla **seconda** copia: ha già visto tutto il documento e il claim della prima, quindi è la lettura più informata.

3. Dove va il preambolo negli altri modelli

Regola unica: **nello stesso turno dello stato, subito prima dello stato**, con lo stesso escaping usato per lo stato (per gli stati in JSON, il claim scritto come stringa JSON).

Modello	Lo stato nel prompt	Dove va il preambolo
KnowLine-4B	testo indentato in un turno utente	prima dello stato
elephant-1	`<state>` JSON indentato `</state>` nel prefisso comune	prima di `<state>`: resta nel prefisso, visto da tutte le domande
StartLux 9B/4B	`Evidence:` + JSON compatto	prima di `Evidence:`
d3-flash, Jeffan-4B	`State:` + JSON	prima di `State:`
OneJev1.6-9B	blocco dello stato nel prefisso comune	prima del blocco
Nimble-9B-v3	turno utente JSON con contesto e schema	campo testuale prima del contesto, se il motore lo accetta; altrimenti Nimble esce dagli esperimenti sul preambolo
pplx-decider-flash	sequenza unica con stato e domande	prima dello stato
Clef-Flash	`STATE:` + JSON compatto	prima di `STATE:`

Si implementa come opzione `preamble=` del renderer di ciascun modello (`evidence/render.py`), che chiama il codice degli autori. Test: con `preamble=None` il prompt è identico token per token a quello del fine-tuning binario; con il preambolo, i token dopo il preambolo sono identici (al più un token di differenza al confine).

4. Scegliere il testo del preambolo

Due candidati:

- **P-A**: `Claim under review: <claim>`
- **P-B**: `The following document will be checked against this claim: <claim>`

Sul dev del fine-tuning binario (`dev_select`), per ciascun modello, si misurano BAcc, AUC ed ECE in sei condizioni:

Condizione	Modello del verdetto	Prompt
B-1D	di partenza (prima del fine-tuning binario)	senza preambolo
B-SAND-A, B-SAND-B	di partenza	P-A, P-B
B-FT	fine-tunato (fine-tuning binario)	senza preambolo
B-FT-SAND-A, B-FT-SAND-B	fine-tunato	P-A, P-B

Si sceglie il testo con cui B-FT-SAND è più vicino a B-FT in AUC; a parità (entro 0,2 punti) il più corto. Lo stesso testo per tutti i modelli.

5. Riusare il verdetto o riaddestrarlo

Risultato su `dev_select`	Cosa fare
B-FT-SAND entro 0,3 punti di BAcc e AUC da B-FT, ECE entro 0,01	il verdetto fine-tunato si usa così com'è; si ricalcola solo la soglia, se si sposta
peggio di così	si riaddestra la LoRA del verdetto con la ricetta del fine-tuning binario e il preambolo nel prompt

6. Dove si scrive la scelta

In `question.json` (il file della domanda congelata del fine-tuning binario) si aggiungono due campi, letti da tutti gli script:

```
"preamble": {"template": "Claim under review: {claim}", "chosen_on": "dev_select"},
"verdict":  {"knowline-4b": "binary_plan", "elephant-1": "retrained_with_preamble"}
```

(Valori di esempio.)

Passo 6 — Dalle frasi ai token, e salvataggio dei vettori

In breve

1. **Rendering** (`evidence/render.py`): per ogni riga si costruisce il prompt con il preambolo (codice degli autori) e si trova, per ogni frase del documento, **quali token del prompt le appartengono** (primo e ultimo). È il dato da cui dipende tutto: se le posizioni sono sbagliate anche di un token, la testa legge i vettori sbagliati.
2. **Estrazione** (`evidence/extract.py`): un forward per riga con gli hook del passo 01; si salvano su disco i vettori di ogni frase. Il modello è congelato: si calcolano una volta e la testa si addestra su di essi in pochi minuti.

1. L'esempio renderizzato

```
{"id": "hover:train:…", "model": "knowline-4b",
 "input_ids": [248045, 846, "…"], "answer_pos": 2371,
 "sent_tok_spans": [[1103, 1117], [1118, 1150], "…"],
 "sent_char_spans": [[0, 61], [62, 140], "…"],
 "label": 1, "gold": [0, 1, 0, "…"],
 "evidence_term": "supervised", "row_weight": 1.0, "omega": 0.5}
```

Campo	Significato
`sent_tok_spans[k]`	primo e ultimo token della frase k nel prompt (KnowLine letto due volte: nella seconda copia)
`sent_char_spans[k]`	inizio e fine della frase k nel documento originale (le `sentences` della riga): servono al server per evidenziare

Campo	Significato
`answer_pos`	la posizione di risposta
`gold[k]`	1 se la frase k è di evidenza (unione degli insiemi alternativi)

2. Il problema: il documento nel prompt non è scritto come l'originale

Per i modelli con lo stato in JSON (elephant-1, StartLux, d3-flash, Jeffan, Nimble, pplx, Clef) virgolette e a capo diventano sequenze di due caratteri (" → \", a capo → \n). Per KnowLine lo stato è testo indentato. Le posizioni delle frasi nel testo originale non coincidono con quelle nel prompt: serve una mappa.

3. Dai token alle frasi

Procedura:

1. costruire il prompt in tre pezzi: prima del documento, documento come appare nel prompt, dopo il documento (si conosce così `doc_start`, dove inizia il documento nel prompt);
2. per il pezzo del documento, `escape_with_map` restituisce il testo trasformato e, per ogni suo carattere, l'indice del carattere originale (`origin`);
3. tokenizzare il prompt con `return_offsets_mapping=True`;
4. `token_char_spans`: per ogni token del documento, i caratteri originali che copre (`tok_char`);
5. `sentence_token_spans`: ogni token va alla frase che contiene il suo ultimo carattere non spazio; per ogni frase, primo e ultimo token.

```
ESC = {'"': '\\"', "\\": "\\\\", "\n": "\\n", "\r": "\\r", "\t": "\\t", "\b": "\\b", "\f": "\\f"}   # come
↪ json.dumps


def escape_with_map(text, escape_lt=False):
    """Il testo come appare in una stringa JSON e, per ogni carattere del testo trasformato, il carattere
    ↪ originale da cui viene."""
    out, origin = [], []
    for i, ch in enumerate(text):
        if ch in ESC:
            piece = ESC[ch]
        elif escape_lt and ch == "<":
            piece = "\\u003c"
        elif ord(ch) < 0x20:
            piece = f"\\u{ord(ch):04x}"
        else:
            piece = ch
        out.append(piece)
        origin.extend([i] * len(piece))
    return "".join(out), origin


def token_char_spans(offsets, doc_start, doc_text_in_prompt_len, origin):
    """offsets: (inizio, fine) di ogni token nel prompt (return_offsets_mapping); doc_start: dove inizia il
    ↪ documento nel prompt;
    origin: per ogni carattere del documento nel prompt, il carattere originale. -> (indici dei token del
↪ documento, tok_char)."""
    doc_end = doc_start + doc_text_in_prompt_len
    idx, spans = [], []
    for k, (a, b) in enumerate(offsets):
        a, b = max(a, doc_start), min(b, doc_end)
        if a >= b:
            continue
        idx.append(k)
        spans.append((origin[a - doc_start], origin[b - 1 - doc_start] + 1))
    return idx, spans


import bisect
```

```python
def sentence_token_spans(tok_char, sentences, text):
    """tok_char: [(inizio, fine)] in caratteri originali dei token del documento (in ordine);
    sentences: [(inizio, fine)] delle frasi (evidence.sentences.split). Un token appartiene alla frase che
    ↪ contiene
    il suo ultimo carattere non spazio. -> [(primo token, ultimo token)] per frase, indici relativi al
    ↪ documento."""
    starts = [s for s, _ in sentences]
    first, last = [None] * len(sentences), [None] * len(sentences)
    for i, (a, b) in enumerate(tok_char):
        c = b - 1
        while c >= a and text[c].isspace():
            c -= 1
        if c < a:
            continue                                    # token di soli spazi: nessuna frase
        k = bisect.bisect_right(starts, c) - 1
        if k < 0 or c >= sentences[k][1]:
            continue                                    # carattere fra due frasi
        first[k] = i if first[k] is None else first[k]
        last[k] = i
    missing = [k for k, f in enumerate(first) if f is None]
    if missing:
        raise ValueError(f"frasi senza token propri: {missing}")
    return list(zip(first, last))
```

Gli indici restituiti sono relativi ai token del documento: si sommano all'indice del primo token del documento nel prompt per avere `sent_tok_spans`.

Provato su un documento con virgolette e a capo dentro un JSON (`Rose "Byrne" is an\nAustralian actress. She starred in films.`): il testo trasformato è identico a `json.dumps`, `\"` torna a `"`, `\n` all'a capo, e le due frasi ricevono i token 0–9 e 10–14.

Casi da gestire:

- **KnowLine** non usa JSON: il documento compare quasi così com'è (con l'indentazione delle righe). `escape_with_map` si sostituisce con la trasformazione del suo `knowline_layout`, registrando allo stesso modo `origin`. Nelle richieste lette due volte si prende la seconda copia del documento.
- **Prefisso e domanda tokenizzati separatamente** (elephant-1, OneJev, Nimble): gli offset si calcolano sui pezzi e si sommano.
- **Token a cavallo di due frasi** (es. `."` in un solo token): va alla frase che contiene il suo ultimo carattere non spazio.
- **Frase senza token propri** (rarissimo: una frase di un carattere fusa in un token): la funzione la segnala; si unisce alla precedente nel rendering.

4. Documenti lunghi

Il filtro di lunghezza del build (02 §3) ha già tolto le righe oltre il contesto. Limiti per fase:

Uso	Lunghezza massima del prompt
Estrazione (fase 1a)	modelli 4B 16.384 token; modelli 9B 8.192 (16.384 se la memoria basta)
Training con il modello in memoria (fasi 1b, 2)	8.192 token
Server	32.768 token

Righe di training oltre il limite: per i positivi si tolgono frasi **non** di evidenza, partendo da quelle più lontane dall'evidenza (le etichette restano valide); per i negativi si taglia la fine. Le righe di dev e test non si tagliano: se sono troppo lunghe si escludono e si contano.

5. Che cosa si salva (fase 1a)

Feature	Forma	Tipo
`q`	4 × D	fp32
`h_end`	n × 4 × D	bf16
`h_mean`	n × 4 × D	bf16
`att`	n × 128	fp16
`n_tok`	n	int16
`answer_logits`	2	fp32

Spazio: 82 KB per frase su KnowLine-4B (4 layer); ~120 GB per le ~28.600 righe delle tre fonti. Se lo spazio non basta, si salvano solo i layer con peso dopo il primo training (o i migliori secondo la sonda, 15 §3).

Formato: `data/evidence/features/<modello>/<set>/shard-00000.safetensors` (1.000 righe per file, tensori concatenati lungo le frasi) più `index.jsonl` (`id`, shard, offset della prima frase, `n`, `gold`, `evidence_term`, pesi). Ogni cartella ha un `meta.json` con modello, revisione dei pesi, preambolo, layer, hash di `sentences.py`, `render.py`, `escape.py`: il training rifiuta vettori calcolati con impostazioni diverse.

6. Comando

```
gpu-run --batch --name ev-extract-knowline -- bash -c \
  'python -m evidence.extract --model knowline-4b \
     --rows dataset/evidence/mixes/ev_head.jsonl dataset/evidence/splits/dev_evid_select.jsonl \
            dataset/evidence/splits/dev_evid_confirm.jsonl dataset/evidence/splits/dev_evid_neg.jsonl \
     --out data/evidence/features/knowline-4b > logs/ev-extract-knowline.out 2>&1'
```

Riprende dagli shard completi; ordina le righe per lunghezza (meno riempimento); batch 1 sopra i 4.096 token.

7. Test

Test	Deve dare
Il testo dei token di ogni frase, decodificato e de-escapato, è la frase (spazi ai bordi a parte)	200 righe su 200, per ogni modello
Documenti con ", \, a capo, tab, <, emoji, testo non latino	posizioni giuste
Senza preambolo, `input_ids` identici al fine-tuning binario	sempre
KnowLine letto due volte: le frasi cadono nella seconda copia	sempre
P(supportato) dai logit salvati = verdetto scelto al passo 5	differenza < 0,005
Stessa riga estratta due volte	vettori identici

Passo 7 — Metodi semplici da battere

In breve

Prima di addestrare la testa si misura quanto ottengono metodi semplici, sullo stesso set (`dev_evid_select`) e con le stesse metriche (10). Servono a due cose: sapere se la testa vale la pena (deve batterli con margine) e capire quanto è difficile il problema sui nostri dati, soprattutto con l'evidenza lontana.

Script: `evidence/baselines.py`. Ogni metodo dà un punteggio per frase; la selezione delle frasi mostrate segue la stessa regola per tutti (§3), così il confronto è sulla qualità del punteggio.

1. I metodi

Nome	Punteggio della frase k	Costo in produzione
E-RAND	casuale	nullo; è il pavimento
E-LEX	BM25 della frase rispetto al claim (`rank_bm25`, parole minuscole senza stopword, sulle frasi del documento)	nullo; è il riferimento principale
E-EMB	somiglianza (coseno) fra l'embedding del claim e quello della frase, con un modello di embedding piccolo (`BAAI/bge-small-en-v1.5`)	un encoder in più
E-ATT	attenzione della posizione di risposta sulla frase (feature `att` del passo 6), combinazione delle migliori coppie (layer, testa)	nullo: legge feature già salvate
E-ABL	gruppo di frasi trovato con l'ablazione del passo 3, con il modello stesso	centinaia di forward per riga: solo riferimento

Che cosa dicono:

- **E-LEX** ed **E-EMB** misurano quanto si ottiene con la sola somiglianza al claim. Non possono trovare una frase "ponte" che non ha parole in comune con il claim (nell'esempio di Nancy Howell, la frase sul seminario): sull'evidenza lontana ci si aspetta che crollino. Se la testa non li batte con margine, l'utente avrebbe quasi lo stesso con un confronto di parole.
- **E-ATT** dice se il modello già "guarda" l'evidenza quando decide, e quanto aggiunge addestrare una testa.
- **E-ABL** è il riferimento "di qualità": fedele al modello per costruzione, ma costa centinaia di forward. Dice quanto ci si avvicina con uno solo.

2. Come si calcola E-ATT

L'attenzione è misurata in 8 layer × 16 teste: non tutte le combinazioni sono utili.

1. Si divide `dev_evid_select` in due metà, A e B, per `doc_id`.
2. Su A, per ognuna delle 128 coppie (layer, testa) e con il punteggio diviso o no per il numero di token della frase (`att / n_tok^β`, $\beta \in \{0; 0{,}5; 1\}$): AUPRC per frase.
3. Si tengono le 8 combinazioni migliori; il punteggio finale è la media dei loro ranghi normalizzati.
4. Si misura su B. Si riporta anche la migliore combinazione singola.

3. Quali frasi si mostrano

I metodi semplici non hanno una soglia propria. Per ciascuno si provano su A:

- le **k migliori**, con k = numero medio di frasi gold;
- una **soglia** sul punteggio, scelta per massimizzare l'F1 per frase (almeno 1, al massimo 5 frasi).

Si riporta la migliore delle due, misurata su B. E-ABL ha già il suo insieme.

4. Comandi

```
python -m evidence.baselines --method rand lex emb --block dev_evid_select
python -m evidence.baselines --method att --model knowline-4b --block dev_evid_select
gpu-run --batch --name ev-eabl-knowline -- bash -c \
  'python -m evidence.baselines --method abl --model knowline-4b --block dev_evid_select --per-source 300 \
     > logs/ev-eabl-knowline.out 2>&1'
```

E-ABL: 300 righe per fonte e per modello.

5. Tabella da compilare (una per modello)

Metodo	F1 frasi HoVer	Insieme completo HoVer	F1 frasi SciFact	AUPRC	F1 con evidenza a ≥ 10 frasi	F1 oltre 1.024 token
E-RAND						
E-LEX						
E-EMB						
E-ATT						
E-ABL						

Risultati in `data/evidence/runs/baselines.json`.

Passo 8 — L'architettura della testa e la loss

In breve

Questo passo descrive per intero la rete da implementare e addestrare, la *testa di evidenza per frase*, e la loss con cui si addestra. Due file:

- `evidence/head.py`: la testa. Riceve i vettori delle frasi e della posizione di risposta e restituisce un punteggio per frase (`s`) e la soglia del documento (`s0`);
- `evidence/losses.py`: la loss.

Ogni parte della testa si accende o spegne con un'opzione: tutti gli esperimenti (11) usano lo stesso codice. Tutto il codice di questo file è stato eseguito su dati finti: forme, gradienti, una riga sola o con righe più lunghe nello stesso batch, valori numerici delle loss, un ciclo di training completo. Parametri e tempi sono misurati.

1. La rete in una figura

Notazione: B = righe nel batch, n = frasi del documento (riempite fino alla riga con più frasi del batch), L = layer letti (4: 19, 23, 27, 31), D = dimensione del modello (2.560 per KnowLine-4B, 4.096 per elephant-1), d = 256.

```
INGRESSI (dal forward del decision model, passo 1)
  q      [B, L, D]     vettore della posizione di risposta, 4 layer
  h_end  [B, n, L, D]  vettore dell'ultimo token di ogni frase, 4 layer
  h_mean [B, n, L, D]  media dei token di ogni frase, 4 layer
  att    [B, n, 128]   attenzione della posizione di risposta sulla frase (8 layer pieni × 16 teste)
  n_tok  [B, n]        numero di token della frase
  mask   [B, n]        1 = frase vera, 0 = riempimento
       |
A. MISCELA DEI LAYER    m(v) = LN( Σ_l softmax(α)_l · v[l] )       stessi pesi α e stessa LN per h_end, h_mean, q
       |
B. VETTORE DELLA FRASE  x[k] = W_h [ m(h_end[k]) ; m(h_mean[k]) ]            [B, n, 256]
                             + W_a [ att[k] ; log n_tok[k] ]
                             + P[k]  (posizione della frase, appresa)  + T_frase
       |
   VETTORE DI RISPOSTA  x0   = W_q m(q) + T_risposta                         [B, 256]
       |
C. COMUNICAZIONE        transformer encoder NON causale, 2 layer, 4 teste, FFN 1.024, pre-norm, dropout 0,1
                        sulla sequenza [x0, x[0], …, x[n−1]]: ogni frase vede tutte le altre e x0
                        → X0 [B, 256],  X [B, n, 256]
       |
D. PUNTEGGI             s[k] = w_o · X[k] + b_o                    [B, n]
                        s0   = w_o · X0  + b_o                     [B]      soglia del documento
       |
```

```
(facoltativo, H-PAIR)   S[k, j] = (U_1 X[k]) · (U_2 X[j])   rango 64     [B, n, n]
                        s[k] ← s[k] + τ · max_{j ≠ k} ( S[k, j] + s[j] )   (s[j] senza gradiente)
        |
SELEZIONE             frasi con s[k] > s0 + δ, al massimo 5, almeno 1 (la migliore) se il verdetto è "supportato"
```

2. Gli ingressi

Tensore	KnowLine-4B	elephant-1	Da dove
`q`	[B, 4, 2.560]	[B, 4, 4.096]	hook sui layer 19, 23, 27, 31, posizione di risposta
`h_end`	[B, n, 4, 2.560]	[B, n, 4, 4.096]	hook, ultimo token di ogni frase
`h_mean`	[B, n, 4, 2.560]	[B, n, 4, 4.096]	hook, media dei token di ogni frase
`att`	[B, n, 128]	[B, n, 128]	righe di attenzione degli 8 layer pieni, sommate per frase
`n_tok`	[B, n]	[B, n]	rendering

n tipico: 9 (SciFact), 46 (HoVer), 68 (WiCE); massimo 512.

3. Strato per strato

A. Miscela dei layer

- Quattro pesi appresi α, che partono da 0 (quindi 25% ciascuno); `softmax(α)` li rende positivi a somma 1. La stessa miscela vale per `h_end`, `h_mean` e `q`, così frasi e posizione di risposta sono rappresentate allo stesso modo.
- Una LayerNorm dopo la miscela: i layer hanno scale molto diverse (l'ultimo ha valori più grandi).
- Perché 4 layer e quali: 15 §3. Dopo il training, i pesi α dicono quali layer servono (se uno ha meno del 10%, si può togliere).

B. Vettore della frase

- `W_h` (2D → 256): ultimo token e media affiancati. L'ultimo token riassume la frase letta fino a lì (in un modello causale vede tutta la frase e ciò che la precede); la media pesa tutti i token allo stesso modo e non dipende dalla punteggiatura finale.
- `W_a` (129 → 256): le 128 feature di attenzione più il logaritmo del numero di token. Una frase lunga raccoglie più attenzione solo perché ha più token: con `log n_tok` la testa può correggere. Durante il training le feature di attenzione si possono azzerare per frase con probabilità 0,1 (dropout), così la testa non dipende da una sola testa di attenzione.
- `P[k]`: vettore appreso della posizione della frase nel documento (0 … 511). Aiuta a trattare titoli e frasi iniziali; l'esperimento H9 controlla che non crei un bias verso inizio o fine.
- `T_frase`, `T_risposta`: vettori di "tipo", per distinguere le frasi da `x0` nella sequenza.

C. Comunicazione fra le frasi

- Transformer encoder **non causale**: ogni elemento della sequenza vede tutti gli altri, in avanti e all'indietro. 2 layer, d = 256, 4 teste, feed-forward 1.024, pre-norm, dropout 0,1. Le frasi di riempimento sono mascherate.
- È il pezzo che risolve l'**evidenza sparsa**: nel modello causale la frase 1 non vede la frase 40; qui sì. Nell'esempio di Nancy Howell, la frase sul seminario (che da sola non nomina Nancy Howell) può "sapere" che più avanti c'è la frase su Nancy Howell e il seminario.
- `x0` porta il punto di vista del verdetto: la posizione di risposta ha visto tutto il prompt e sa su cosa si è basata la decisione.

D. Punteggi e soglia

- Un'unica uscita lineare `w_o` (256 → 1) dà `s[k]` per ogni frase e, applicata a `X0`, la soglia `s0`. Stessa uscita per entrambi: punteggi e soglia sono sulla stessa scala, e `s[k] − s0` ha senso.
- Una frase è evidenza se `s[k] > s0`. La soglia è diversa per ogni documento: con un claim che richiede 3 frasi, `X0` (che sa quante parti servono) può abbassarla per farle passare tutte.

Facoltativo: punteggio per coppie (H-PAIR)

- Per ogni coppia di frasi un punteggio bilineare a rango 64; ogni frase riceve, moltiplicato per τ (appreso, parte da 0,1), il massimo fra "punteggio della coppia + punteggio dell'altra frase" sulle altre frasi. Premia una frase quando c'è un'altra frase con cui "fa coppia".
- Si usa solo se, dopo gli esperimenti principali, l'insieme completo su HoVer resta sotto 0,65.

4. Parametri e costo (misurati)

Componente	KnowLine-4B	elephant-1
A. miscela e normalizzazione	5.124	8.196
B. `W_h` (2D → 256)	1.310.976	2.097.408
B. `W_a` (129 → 256)	33.280	33.280
B. posizione (512 frasi) e tipi	131.584	131.584
`W_q` (D → 256)	655.616	1.048.832
C. transformer	1.579.520	1.579.520
D. uscita	257	257
Totale (senza coppie)	**3.716.357** (15 MB in fp32)	**4.899.077** (19,6 MB)
Coppie (solo H-PAIR)	+ 32.769	+ 32.769

Tempo misurato della testa (GPU locale RTX 5090 Laptop, fp32, una riga): **0,67 ms** con 46 frasi, 0,83 ms con 200, 1,55 ms con 500. Il forward del modello su un documento di ~2.000 token costa decine o centinaia di millisecondi: la testa aggiunge meno dell'1–2%.

5. Il codice della testa

```
import torch
import torch.nn as nn

class EvidenceHead(nn.Module):
    def __init__(self, d_model, n_mix, n_att, d=256, layers=2, heads=4, max_sent=512,
                 use_mix=True, use_mean=True, use_att=True, use_pos=True, use_pair=False, pair_rank=64,
 ↪  att_dropout=0.1):
        super().__init__()
        self.use_mix, self.use_mean, self.use_att, self.use_pos, self.use_pair = use_mix, use_mean, use_att,
         ↪  use_pos, use_pair
        self.att_dropout = att_dropout
        self.mix = nn.Parameter(torch.zeros(n_mix))                    # pesi dei layer (punto 1)
        self.ln = nn.LayerNorm(d_model)                                 # i layer hanno scale diverse
        self.proj_h = nn.Linear(d_model * (2 if use_mean else 1), d)    # [ultimo token ; media] → 256
        self.proj_q = nn.Linear(d_model, d)                             # q → 256
        self.proj_a = nn.Linear(n_att + 1, d)                          # attenzione + log(numero di token) → 256
        self.pos = nn.Embedding(max_sent, d)                            # posizione della frase
        self.kind = nn.Embedding(2, d)                                  # 0 = x_0, 1 = frase
        enc = nn.TransformerEncoderLayer(d, heads, 4 * d, dropout=0.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(enc, layers)               # tutti vedono tutti (punto 4)
        self.out = nn.Linear(d, 1)                                      # punteggio (punto 5)
        self.pair = nn.ModuleList([nn.Linear(d, pair_rank, bias=False), nn.Linear(d, pair_rank, bias=False)])
        self.pair_tau = nn.Parameter(torch.tensor(0.1))

    def mixed(self, x):                                                 # x: [..., n_mix, D]
        if not self.use_mix:
```

```
            return self.ln(x[..., -1, :])                                  # solo l'ultimo layer
        return self.ln((self.mix.softmax(0)[:, None] * x).sum(-2))

    def forward(self, q, h_end, h_mean, att, n_tok, sent_mask):
        """q: [B, L, D]; h_end, h_mean: [B, N, L, D]; att: [B, N, n_att]; n_tok, sent_mask: [B, N] (1 = frase
        ↪  vera).
        Restituisce s: [B, N], s0: [B], S: [B, N, N] o None."""
        h = self.mixed(h_end)
        if self.use_mean:
            h = torch.cat([h, self.mixed(h_mean)], -1)
        x_h = self.proj_h(h) + self.kind.weight[1]
        if self.use_pos:
            x_h = x_h + self.pos(torch.arange(h.size(1), device=h.device))
        if self.use_att:
            att = att.float()
            if self.training and self.att_dropout:                         # intere, per frase: non dipendere da
            ↪  una sola testa
                att = att * (torch.rand_like(att[..., :1]) >= self.att_dropout)
            x_h = x_h + self.proj_a(torch.cat([att, n_tok.clamp_min(1).float().log()[..., None]], -1))
        x_0 = (self.proj_q(self.mixed(q)) + self.kind.weight[0])[:, None]
        pad = torch.cat([torch.zeros_like(sent_mask[:, :1]), 1 - sent_mask], 1).bool()
        X = self.encoder(torch.cat([x_0, x_h], 1), src_key_padding_mask=pad)
        o = self.out(X)[..., 0]
        s, s0, S = o[:, 1:], o[:, 0], None
        if self.use_pair:
            Xs = X[:, 1:]
            S = self.pair[0](Xs) @ self.pair[1](Xs).transpose(1, 2)    # punteggio di ogni coppia
            invalid = ~sent_mask.bool()[:, None, :] | torch.eye(S.size(1), dtype=torch.bool, device=S.device)
            support = (S + s.detach()[:, None, :]).masked_fill(invalid, float("-inf")).amax(-1)
            s = s + self.pair_tau * support.nan_to_num(neginf=0.0)      # nessun partner: nessun contributo
        return s, s0, S
```

Configurazione di partenza dell'esperimento **H0**: `use_mix=False, use_mean=False, use_att=False, use_pair=False` (solo l'ultimo layer, solo l'ultimo token, niente attenzione), loss di riferimento (§6.2), soglia fissa. Gli esperimenti (11) accendono un pezzo alla volta.

6. La loss

6.1 I tipi di riga

Ogni riga del mix ha un `evidence_term` (04 §3):

Tipo	Righe	Che cosa chiede la loss
`supervised`	positivi con evidenza nota	le frasi gold sopra la soglia, le altre sotto
`mil_neg`	negativi	**nessuna** frase sopra la soglia: una frase che somiglia al claim ma non lo dimostra non va mostrata
`mil_pos`	positivi senza evidenza nota (solo esperimento H-MIL)	almeno una frase sopra la soglia, ma poche
`none`	righe senza evidenza utilizzabile (WiCE parzialmente supportati)	niente

Due pesi per riga: `row_weight` (quanto conta la riga) e ω (quanto pesano le frasi non gold: basso dove le annotazioni sono incomplete).

6.2 Loss di riferimento (H0): per frase, con soglia fissa

Ogni frase è un sì/no indipendente ("è evidenza?"), con due correzioni:

- le frasi gold sono poche (2 su 50): senza correzione la testa imparerebbe a dire sempre "no", quindi pesano di più (α, ~10);

- un termine di **ordinamento**: ogni frase gold deve superare di un margine la migliore frase non gold.

In uso si mostrano le frasi con σ(s) > τ_e, una soglia fissa scelta sul dev. Solo righe `supervised`.

```
import torch
import torch.nn.functional as F

def ref_loss(s, gold, sent_mask, omega, alpha=10.0, gamma=0.5, margin=1.0):
    """s, gold, sent_mask, omega: [B, N] (omega = peso delle frasi non di evidenza). Restituisce [B]."""
    bce = F.binary_cross_entropy_with_logits(s, gold.float(), reduction="none")
    w = torch.where(gold > 0, torch.full_like(s, alpha), omega) * sent_mask
    l_bce = (bce * w).sum(1) / sent_mask.sum(1)
    best_neg = s.masked_fill((gold > 0) | (sent_mask == 0), float("-inf")).amax(1, keepdim=True)
    rank = F.softplus(margin - s + best_neg).nan_to_num(posinf=0.0) * (gold > 0)
    l_rank = rank.sum(1) / (gold > 0).sum(1).clamp(min=1)
    return l_bce + gamma * l_rank
```

Esempio (test): 40 frasi, evidenza {1, 40}, α = 10, tutte con probabilità 0,02: parte per frase = **1,975**, quasi tutta dalle due frasi gold (10 · 2 · 3,91 / 40): la loss spinge su quelle due.

6.3 Loss con soglia del documento (ZLPR + MIL)

La regola è: **una frase è evidenza se e solo se `s[k] > s0`**. La loss (ZLPR di Su et al., con la soglia appresa come "classe soglia") spinge:

- in `supervised`: le frasi gold sopra `s0`, tutte le altre sotto;
- in `mil_neg`: tutte le frasi sotto `s0`;
- in `mil_pos`: almeno una frase sopra `s0` (massimo "morbido", LSE), più un termine di sparsità.

```
supervised:  log(1 + Σ_{i ∉ G} ω_i · e^(s_i − s0))  +  log(1 + Σ_{j ∈ G} e^(s0 − s_j))
mil_neg:     0,3 · log(1 + Σ_i e^(s_i − s0))
mil_pos:     0,3 · [ log(1 + e^(s0 − LSE_j s_j))  +  0,1 · media_i σ(s_i − s0) ]
none:        0
```

Il primo termine di `supervised` cresce se qualche frase non gold sta sopra la soglia, il secondo se qualche frase gold sta sotto. Rispetto alla loss di riferimento: nessuna soglia fissa da tarare, numero di frasi mostrate adattato al documento, e i negativi entrano nel training.

```
def zlpr_loss(s, s0, gold, sent_mask, omega, term, mu_neg=0.3, mu_pos=0.3, rho=0.1):
    """term: [B] con 0 = supervised, 1 = mil_neg, 2 = mil_pos, 3 = nessun termine. Restituisce [B]."""
    ninf = torch.finfo(s.dtype).min
    real = sent_mask.bool()
    d = s - s0[:, None]
    is_neg = (gold == 0) & real
    l_neg = F.softplus(torch.logsumexp((d + omega.clamp_min(1e-6).log()).masked_fill(~is_neg, ninf), 1))
    l_pos = F.softplus(torch.logsumexp((-d).masked_fill(~(gold > 0), ninf), 1)) * (gold.sum(1) > 0)
    supervised = l_neg + l_pos
    lse = torch.logsumexp(d.masked_fill(~real, ninf), 1)
    mil_neg = mu_neg * F.softplus(lse)
    spars = (torch.sigmoid(d) * sent_mask).sum(1) / sent_mask.sum(1)
    mil_pos = mu_pos * (F.softplus(-lse) + rho * spars)
    out = torch.stack([supervised, mil_neg, mil_pos, torch.zeros_like(lse)], 1)
    return out.gather(1, term[:, None]).squeeze(1)
```

Esempio numerico (test): 40 frasi, evidenza = frasi 1 e 40, `s0 = 0`, ω = 1:

	frase 1	frase 40	una frase simile al claim	altre 37	Loss
prima	2,0	−0,5 (sotto soglia: verrebbe persa)	0,5 (sopra soglia: verrebbe mostrata)	−3	**2,526**
dopo	2,0	1,5	−1,0	−3	**1,473**

La loss scende quando la frase 40 sale sopra la soglia e la frase simile scende sotto.

Rischio del termine `mil_neg`: `x0` conosce il verdetto, quindi la testa può imparare "se il verdetto è no, alza la soglia". È coerente con l'obiettivo (l'evidenza si mostra solo sui supportati), ma sui positivi incerti (P appena sopra la soglia del verdetto) la testa può spegnersi: si controlla la recall in quella fascia (09 §3) e si abbassa il peso 0,3 se serve.

6.4 Termine per le coppie (solo H-PAIR)

Nelle righe con 2 o più frasi gold, il punteggio di coppia deve essere alto per le coppie di frasi gold e basso fra una frase gold e le 5 frasi non gold più "vicine". Peso 0,2.

```
def pair_loss(S, gold, sent_mask, n_hard=5):
    g = gold.bool() & sent_mask.bool()
    pos_pairs = g[:, :, None] & g[:, None, :] & ~torch.eye(S.size(1), dtype=torch.bool, device=S.device)
    l_pos = F.binary_cross_entropy_with_logits(S[pos_pairs], torch.ones_like(S[pos_pairs]))
    neg_cand = S.masked_fill(~(g[:, :, None] & (~g & sent_mask.bool())[:, None, :]), float("-inf"))
    hard = neg_cand.topk(min(n_hard, S.size(1)), -1).values
    hard = hard[torch.isfinite(hard)]
    l_neg = F.binary_cross_entropy_with_logits(hard, torch.zeros_like(hard))
    return l_pos + l_neg
```

7. La selezione delle frasi

```
def select(s, s0, delta=0.0, max_shown=5, min_shown=1):
    """Frasi mostrate: s[k] > s0 + delta, al massimo max_shown (le più alte), almeno min_shown (la migliore)."""
    order = sorted(range(len(s)), key=lambda k: s[k], reverse=True)
    chosen = [k for k in order if s[k] > s0 + delta][:max_shown]
    return sorted(chosen or order[:min_shown])
```

δ si sceglie sul dev (09 §2); con la loss di riferimento la regola è σ(s) > τ_e.

8. Test (`tests/test_evidence_head.py`)

- forme dei tensori, frasi di riempimento ignorate, riga con una sola frase, con e senza coppie;
- i punteggi di una riga non cambiano se nel batch c'è una riga con più frasi;
- gli esempi numerici delle loss (tolleranza 1e-3): 1,975; 2,526; 1,473; `mil_neg` con tutte le frasi molto sotto soglia ≈ 0; una riga `none` dà loss e gradiente zero;
- con tutte le opzioni spente la testa è la configurazione H0.

Passo 9 — Training

In breve

Si addestra la testa in tre modi, dal più semplice al più costoso; si passa al successivo solo se il precedente non basta.

Fase	Il decision model	Ingresso della testa	Quando
1a	congelato, non in memoria	vettori delle frasi salvati al passo 6	quasi tutti gli esperimenti: ogni prova dura minuti
1b	congelato, in memoria	vettori ricalcolati a ogni passo	solo per l'esperimento che ha bisogno dei vettori di tutti i token (H2d, media pesata dei token)
2	addestrato anche lui, con una LoRA	come 1a, con gradiente nel modello	solo se dopo gli esperimenti gli obiettivi non sono raggiunti

Nelle fasi 1a e 1b il verdetto **non può cambiare**: il modello non viene toccato.

Ogni prova scrive in `data/evidence/runs/<modello>/<id>/`: `config.json` (opzioni della testa, della loss, iperparametri, hash delle feature e del mix), `head.safetensors`, `log.jsonl`, `dev_metrics.json`.

1. Fase 1a: il ciclo di training

Ogni riga del mix porta: le feature salvate (`q`, `h_end`, `h_mean`, `att`, `n_tok`), le etichette delle frasi (`gold`), il tipo di loss (`evidence_term`), ω e il peso della riga.

1. **Batch per lunghezza**: le righe si ordinano per numero di frasi e si raggruppano a 64, così il riempimento è minimo; l'ordine dei batch è casuale e cambia a ogni epoca.
2. **Riempimento**: ogni batch si porta alla riga con più frasi; `mask` segna le frasi vere. La testa è invariante al riempimento (verificato: differenza ~1e-7).
3. **Loss** (08 §6.3) per riga, moltiplicata per il peso della riga; somma divisa per la somma dei pesi delle righe con loss.
4. **Ottimizzazione**: AdamW, clipping del gradiente a 1, learning rate con riscaldamento lineare (primo 5% dei passi) e discesa a coseno.
5. **A fine epoca**: valutazione su `dev_evid_select` con la selezione e le metriche del passo 10; si tiene il checkpoint migliore; ci si ferma dopo 2 epoche senza miglioramento.

```
import math
import random

import torch

from head import EvidenceHead
from losses import zlpr_loss

TERMS = {"supervised": 0, "mil_neg": 1, "mil_pos": 2, "none": 3}


def collate(rows, device):
    """rows: dict con q [L, D], h_end [n, L, D], h_mean [n, L, D], att [n, 128], n_tok [n], gold [n] (0/1),
    omega, evidence_term, row_weight. -> tensori riempiti fino alla riga con più frasi del batch."""
    n = max(r["h_end"].shape[0] for r in rows)
    B, (L, D) = len(rows), rows[0]["q"].shape
    h_end, h_mean = torch.zeros(B, n, L, D), torch.zeros(B, n, L, D)
    att, n_tok = torch.zeros(B, n, rows[0]["att"].shape[1]), torch.ones(B, n)
    gold, mask, omega = torch.zeros(B, n), torch.zeros(B, n), torch.ones(B, n)
    for i, r in enumerate(rows):
        k = r["h_end"].shape[0]
        h_end[i, :k], h_mean[i, :k], att[i, :k] = r["h_end"].float(), r["h_mean"].float(), r["att"].float()
        n_tok[i, :k], gold[i, :k], mask[i, :k], omega[i, :k] = r["n_tok"].float(), r["gold"].float(), 1,
↪   r["omega"]
    q = torch.stack([r["q"].float() for r in rows])
    term = torch.tensor([TERMS[r["evidence_term"]] for r in rows])
    weight = torch.tensor([r["row_weight"] for r in rows], dtype=torch.float)
    return [t.to(device) for t in (q, h_end, h_mean, att, n_tok, mask, gold, omega, term, weight)]


def batches(rows, size, seed):
    order = sorted(range(len(rows)), key=lambda i: rows[i]["h_end"].shape[0])
    groups = [order[i:i + size] for i in range(0, len(order), size)]
    random.Random(seed).shuffle(groups)
    return groups


def batch_loss(model, rows, device):
    q, h_end, h_mean, att, n_tok, mask, gold, omega, term, weight = collate(rows, device)
    s, s0, _ = model(q, h_end, h_mean, att, n_tok, mask)
    loss = zlpr_loss(s, s0, gold, mask, omega, term) * weight
    active = (term != TERMS["none"]).float() * weight
    return loss.sum() / active.sum().clamp_min(1e-6)


def train(rows, dev_rows, d_model, epochs=10, batch=64, lr=1e-3, seed=13, device="cuda", patience=2,
↪   evaluate=None,
          **head_kw):
```

```
    torch.manual_seed(seed)
    model = EvidenceHead(d_model=d_model, n_mix=rows[0]["q"].shape[0], n_att=rows[0]["att"].shape[1],
↪   **head_kw).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    steps = epochs * math.ceil(len(rows) / batch)
    sched = torch.optim.lr_scheduler.LambdaLR(
        opt, lambda k: min(1.0, k / max(1, 0.05 * steps)) * 0.5 * (1 + math.cos(math.pi * min(k, steps) / steps)))
    best, best_state, bad = -1.0, None, 0
    for epoch in range(epochs):
        model.train()
        for idx in batches(rows, batch, seed + epoch):
            loss = batch_loss(model, [rows[i] for i in idx], device)
            opt.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            sched.step()
        score = evaluate(model, dev_rows) if evaluate else -loss.item()
        print(f"epoch {epoch}  loss {loss.item():.4f}  dev {score:.4f}")
        if score > best:
            best, best_state, bad = score, {k: v.detach().clone() for k, v in model.state_dict().items()}, 0
        else:
            bad += 1
            if bad >= patience:
                break
    model.load_state_dict(best_state)
    return model, best
```

Questo ciclo è stato eseguito su feature finte (300 righe di 10–80 frasi, segnale nascosto nelle frasi gold, un quarto di negativi): in 5 epoche arriva a F1 = 1 sul dev finto, con arresto anticipato e ripristino del checkpoint migliore.

Nel codice vero le righe si leggono dagli shard del passo 6 (`evidence/feature_dataset.py`, che restituisce gli stessi dizionari) e la funzione `evaluate` usa `select` e le metriche del passo 10.

Iperparametro	Valore
Learning rate	1e-3, riscaldamento 5%, coseno
Weight decay	0,01
Batch	64 righe raggruppate per lunghezza
Epoche	fino a 10, arresto dopo 2 senza miglioramento
Dropout	0,1 nel transformer; 0,1 sulle feature di attenzione (intere, per frase)
Criterio del checkpoint	media di F1 per frase e insieme completo su `dev_evid_select`
Seme	13 (3 semi per la configurazione finale: 13, 14, 15)

```
python -m evidence.train_head --phase 1a --model knowline-4b --features data/evidence/features/knowline-4b \
    --mix ev_head --config configs/h0.json --run h0
```

La fase 1a non ha bisogno del modello in memoria: gira su qualunque GPU, anche su CPU.

Prima della prima prova: la testa deve imparare a memoria 64 righe (F1 ≈ 1 su quelle righe). Se non ci riesce, c'è un errore nelle posizioni delle frasi, nelle etichette o nel codice.

2. La regola di selezione delle frasi

Si fissa dopo il training, su `dev_evid_select`, solo sulle righe supportate che il verdetto giudica supportate:

Loss usata	Regola	Cosa si sceglie
loss di riferimento (H0)	$\sigma(s[k]) > \tau_e$	τ_e che dà l'F1 per frase migliore

Loss usata	Regola	Cosa si sceglie
loss con soglia del documento	s[k] > s0 + δ	δ fra −1 e 1, passo 0,1; si tiene δ ≠ 0 solo se guadagna almeno 1 punto di F1

Sempre almeno 1 frase (la migliore) se il verdetto è “supportato” e al massimo 5. Conferma su `dev_evid_confirm`: F1 non più di 1 punto sotto. I valori vanno in `decision.json`.

3. Che cosa guardare durante il training

Cosa	Su	Segnale di problema	Cosa fare
F1 e insieme completo	`dev_evid_select`	non superano E-LEX dopo 2 epoche	controllare le posizioni delle frasi (06 §7) prima di cambiare la testa
F1 per distanza fra le frasi gold	righe con evidenza lontana	crollo oltre 10 frasi	più righe spostate; transformer a 4 layer; H-PAIR
F1 per distanza dall’inizio del documento	`dev_evid_select`	crollo oltre 1.024 o 4.096 token	più righe lontane; attenzione come feature
Frasi mostrate per riga	`dev_evid_select`	molte più o molte meno delle frasi gold	δ, ω
Negativi con frasi sopra soglia	`dev_evid_neg`	oltre il 30%	peso di `mil_neg` più alto
Recall sui positivi incerti (P appena sopra la soglia del verdetto, fino a +0,15)	`dev_evid_select`	molto sotto la media	peso di `mil_neg` più basso
Pesi α dei layer	testa	uno sotto il 10%	togliere quel layer (meno spazio, meno costo)

4. Fase 1b: con il modello in memoria

Il modello è caricato e congelato; a ogni passo si rifà il forward (senza gradiente) per avere i vettori. Serve per un solo esperimento:

- **H2d**: invece della media fissa dei token di una frase, una media pesata in cui la testa decide quali token contano (per esempio il nome proprio dentro una frase lunga). Per ogni frase: proiezione `d → 256` dei vettori dei suoi token, punteggio di ogni token contro una query `u = W_u [c ; q]` (`c` = media dei token del preambolo), softmax dentro la frase, somma pesata. Richiede i vettori di tutti i token, che non si salvano su disco.

Impostazione	Gruppo 4B	Gruppo 9B
GPU	24 GB	45 GB
Righe per passo	4–8 (accumulando fino a 32)	1–2 (accumulando fino a 32)
Lunghezza massima	8.192 token	8.192 token
Learning rate	5e-4	5e-4
Partenza	la testa migliore della fase 1a	idem

5. Fase 2: training anche del modello

Solo se gli obiettivi non sono raggiunti dopo gli esperimenti della fase 1. Il modello del verdetto è il decision model con la LoRA del fine-tuning binario (rango 16 sui moduli di attenzione, DeltaNet e MLP di Qwen3.5). Si continua ad addestrare quella LoRA insieme alla testa.

Impostazione	Valore
Partenza	LoRA del verdetto + testa migliore della fase 1
Learning rate	LoRA 5e-6 – 1e-5 (basso: il verdetto è già addestrato); testa 2e-4
Loss	loss del verdetto (cross-entropy sui due punteggi sì/no, su tutte le righe) + 0,3 × loss di evidenza
Dati	i dati del fine-tuning binario più le righe con evidenza
Epoche	1
GPU	4B: 24 GB; 9B: 45 GB

Il verdetto ora può cambiare, quindi:

- prima del primo passo si controlla che sia identico a quello di partenza;
- a ogni checkpoint si misurano BAcc, AUC ed ECE sul dev del fine-tuning binario; un checkpoint si accetta solo se BAcc resta entro 0,3 punti ed ECE entro 0,01;
- se la calibrazione si sposta, si aggiunge un termine che tiene le probabilità vicine a quelle del verdetto di partenza: $\beta \cdot \mathrm{KL}(p_{\text{partenza}} \,\|\, p)$, $\beta = 0{,}1$;
- se il termine `mil_neg` sposta la calibrazione, lo si applica senza gradiente verso il modello (agisce solo sulla testa).

6. Test prima di ogni nuova configurazione

Test	Atteso
Imparare a memoria 64 righe	F1 ≈ 1 sul training
H0 riprodotto con le opzioni spente	stessi numeri (stesso seme)
Righe `none`	loss e gradiente zero
Batch con righe di lunghezze molto diverse	stessi punteggi di una riga alla volta (differenza < 1e-4)

Passo 10 — Valutazione

In breve

Per ogni modello si misurano tre cose:

1. il **verdetto** non è cambiato;
2. le **frasi mostrate** sono quelle giuste, anche quando sono lontane fra loro o lontane dall'inizio del documento;
3. la **latenza** resta quella di un decision model.

Script: `evidence/eval_evidence.py` (qualità), `evidence/latency.py` (velocità).

1. Su quali righe

L'evidenza si misura sulle righe supportate **che il modello giudica supportate**: sono le sole in cui il sistema mostra frasi. A parte si riporta quante righe supportate il modello giudica non supportate (e quindi restano senza evidenza). Per confrontare modelli diversi si usano anche le sole righe giudicate supportate da tutti.

2. Metriche dell’evidenza

Metrica	Che cosa misura
Precisione, recall, F1 per frase	quante frasi mostrate sono giuste, quante giuste sono state mostrate (rispetto all’unione degli insiemi gold)
Insieme completo	in quante righe sono state mostrate **tutte** le frasi di almeno un insieme gold. Per l’evidenza su più frasi è la metrica che conta: metà evidenza non convince
Corrispondenza esatta	frasi mostrate = uno degli insiemi gold
AUPRC per frase	quanto bene i punteggi ordinano le frasi, indipendentemente dalla soglia
Errore sul numero	di quante frasi in più o in meno rispetto alle frasi gold
F1 per distanza fra le frasi gold	vicine / 2–9 / 10–19 / ≥ 20 frasi fra la prima e l’ultima frase gold
F1 per distanza dall’inizio del documento	ultima frase gold a < 1.024 / 1.024–4.096 / > 4.096 token
Sufficienza	in quante righe il documento ridotto alle sole frasi mostrate fa ancora dire “supportato” al modello. Non dipende dalle annotazioni, che possono essere incomplete
Necessità	in quante righe il documento senza le frasi mostrate fa dire “non supportato” (si riporta solo: se l’evidenza è ripetuta nel documento può fallire anche quando è giusta)
Negativi accesi	su `dev_evid_neg`: in quanti claim non supportati almeno una frase supera la soglia

Sufficienza e necessità richiedono 2 forward per riga, con lo stesso modello e lo stesso prompt di produzione: si calcolano solo in valutazione.

```python
from sklearn.metrics import average_precision_score


def select(s, s0, delta=0.0, max_shown=5, min_shown=1):
    """Frasi mostrate: s[k] > s0 + delta, al massimo max_shown (le più alte), almeno min_shown (la migliore)."""
    order = sorted(range(len(s)), key=lambda k: s[k], reverse=True)
    chosen = [k for k in order if s[k] > s0 + delta][:max_shown]
    return sorted(chosen or order[:min_shown])


def sentence_metrics(shown, evidence_sets):
    """shown: indici delle frasi mostrate; evidence_sets: insiemi alternativi di indici gold.
    -> precisione, recall, F1 rispetto all'unione; insieme completo se uno degli insiemi è tutto mostrato."""
    gold = set().union(*map(set, evidence_sets))
    shown = set(shown)
    tp = len(shown & gold)
    p = tp / len(shown) if shown else 0.0
    r = tp / len(gold) if gold else 0.0
    return {"precision": p, "recall": r, "f1": 2 * p * r / (p + r) if p + r else 0.0,
            "full_set": any(set(e) <= shown for e in evidence_sets), "exact": any(set(e) == shown for e in
            ↪  evidence_sets)}


def auprc(scores_per_row, gold_per_row):
    """AUPRC per frase su tutte le righe (indipendente dalla soglia)."""
    y = [int(k in g) for s, g in zip(scores_per_row, gold_per_row) for k in range(len(s))]
    x = [v for s in scores_per_row for v in s]
    return average_precision_score(y, x)
```

Esempi (eseguiti): frasi mostrate {1, 4}, insiemi gold {1, 4} e {7} → precisione 1, recall 0,67 (rispetto

all'unione {1, 4, 7}), F1 0,8, insieme completo sì, esatto sì. Frase mostrata {1}, insieme gold {1, 4} → recall 0,5, F1 0,67, insieme completo no.

3. Metriche del verdetto

BAcc, AUC ed ECE sul dev e sui test del fine-tuning binario, con il prompt con preambolo, confrontate con il verdetto scelto al passo 5. Nelle fasi 1a e 1b devono coincidere: se no, c'è un errore nella pipeline.

4. Prove di comportamento

Costruite dal test con le trasformazioni del passo 3:

Prova	Righe	Risultato atteso
Evidenza spostata all'inizio / a metà / alla fine	3 × 150	stesse frasi mostrate in ≥ 90% delle righe
Evidenza spostata oltre 1.024 e oltre 4.096 token	2 × 150	idem
Frasi di disturbo aggiunte	200	nessuna frase di disturbo mostrata in ≥ 95%
Frase di evidenza ripetuta altrove	150	almeno una delle due copie mostrata, sempre
Frase di evidenza riscritta senza parole in comune col claim	150	mostrata in ≥ 80% (E-LEX fallisce qui)
Coppia quasi identica: positivo e negativo con un fatto tolto	200 coppie	positivo: frasi giuste; negativo: "non supportato" e nessuna frase sopra soglia

5. Latenza

Stesso server (12), stessa GPU, 1.000 richieste reali di lunghezze diverse (le prime 50 non si contano):

	Prompt	Uscita
A	senza preambolo	solo verdetto
B	con preambolo	solo verdetto
C	con preambolo	verdetto + evidenza

Obiettivo: C al massimo +3% rispetto a B (modelli 4B: +3% o +2 ms). La differenza B − A è il costo del preambolo e si riporta a parte. Si misurano anche, separatamente, la divisione in frasi (CPU), il riassunto delle frasi negli hook e la testa (0,7–1,6 ms misurati per 46–500 frasi su RTX 5090 Laptop).

6. Comandi

```
python -m evidence.eval_evidence --model knowline-4b --run data/evidence/runs/knowline-4b/<id> --block
↪  dev_evid_select
# test: una volta sola, a configurazione congelata (passo 12)
gpu-run --batch --name ev-test -- bash -c \
  'python -m evidence.eval_evidence --model knowline-4b --release <rilascio> \
     --block test_evid_id test_evid_ood test_evid_sparse --sufficiency > logs/ev-test.out 2>&1'
gpu-run --batch --name ev-latency -- bash -c \
  'python -m evidence.latency --model knowline-4b --release <rilascio> > logs/ev-latency.out 2>&1'
```

Lo script salva `metrics.json` e `predictions.jsonl` (per ogni riga: punteggi, soglia, frasi mostrate, frasi gold), così le metriche si ricalcolano senza rifare i forward.

7. Report finale (uno per modello)

Set	Metrica	E-LEX	E-ATT	E-ABL	Testa
test_evid_id	F1 / insieme completo				
test_evid_ood	F1 / insieme completo				
test_evid_sparse	F1 a ≥ 20 frasi / oltre 4.096 token				
tutti	sufficienza				
dev_evid_neg	negativi accesi				
verdetto	BAcc (uguale al fine-tuning binario?)	—	—	—	
latenza	p50 / p95, C contro B				

Passo 11 — Esperimenti

In breve

Si parte da una testa semplice (H0) e si aggiunge **un pezzo alla volta**, tenendolo solo se migliora. Tutto si fa su **KnowLine-4B** con i vettori salvati (fase 1a): ogni prova è un training della sola testa, di pochi minuti. Alla fine elephant-1 e gli altri decision model ripetono la configurazione migliore.

Regole

1. Una modifica per prova, con il suo `configs/<id>.json` (opzioni della testa e della loss).
2. Si decide su `dev_evid_select` guardando quattro numeri: F1 per frase, insieme completo, F1 con evidenza a ≥ 10 frasi di distanza, F1 con evidenza oltre 1.024 token. In più, i negativi accesi su `dev_evid_neg`.
3. **Una modifica si tiene** se aumenta l'F1 di almeno 1 punto o l'insieme completo di almeno 2, senza far scendere gli altri numeri di più di 1 punto. Se aggiunge dati da leggere dal modello, deve anche restare nel limite di latenza.
4. A parità di risultato, la versione più semplice.
5. Guadagni sotto 2 punti si ripetono con 2 semi in più prima di tenerli.

La testa di partenza: H0

Solo l'ultimo layer (31), solo il vettore dell'ultimo token di ogni frase, niente attenzione, 2 layer di transformer fra le frasi, loss di riferimento con soglia fissa (08 §6.2), solo righe positive.

Prima domanda: H0 batte E-LEX ed E-ATT? Se no, prima di provare altro si cerca un errore nelle posizioni o nei vettori (06 §7).

Esperimenti principali (in quest'ordine)

Nome	Che cosa si cambia	Che cosa si guarda
H-LOSS	loss con soglia del documento (ZLPR, 08 §6.3) invece della soglia fissa	insieme completo; numero di frasi mostrate
H-MIL	si aggiungono i claim non supportati al training (`mil_neg`), con peso 0,3 e 1	negativi accesi; che non si perdano frasi nei positivi incerti
H2a	vettori da 4 layer (19, 23, 27, 31) invece che dal solo ultimo; poi solo i 2 con più peso	F1; pesi α dei layer
H2b	si aggiunge l'attenzione ricevuta da ogni frase	F1 con evidenza lontana; frasi "ponte"

Nome	Che cosa si cambia	Che cosa si guarda
H2c	si aggiunge la media dei token della frase	F1 sulle frasi lunghe (oltre 25 parole)

Controlli sulla configurazione migliore

Ognuno toglie o cambia un pezzo, per capire quanto conta:

Nome	Che cosa si cambia	Che cosa dice
H3a	niente transformer fra le frasi: ogni frase ha un punteggio da sola	se le frasi devono comunicare. Atteso: crollo su HoVer e sull'evidenza lontana; se non crolla, si può semplificare
H3b	punteggio = somiglianza bilineare fra `x0` e la frase	se basta una testa più semplice
H3c	1 o 4 layer di transformer invece di 2	quanta capacità serve
H4	vettori calcolati senza preambolo	quanto aiuta il preambolo; si ripete su elephant-1, che è bidirezionale. Se aiuta poco, si toglie
H5	niente righe con evidenza spostata nel training (passo 3)	se servono. Atteso: calo sull'evidenza lontana
H7	ω = 0,3 / 0,5 / 1 per le frasi non gold	come trattare annotazioni incomplete
H8	etichette: solo annotate / + generate / + ricavate col modello / + positivi senza evidenza (`mil_pos`) al posto delle ricavate	quanto valgono i dati del passo 3
H9	niente vettore di posizione della frase	se la testa preferisce troppo l'inizio o la fine
H12	peso delle righe HoVer `db_split`: 0 / 0,5 / 1	se le righe recuperate dal db aiutano o fanno rumore

Esperimenti da fare solo se serve

Nome	Quando	Che cosa
H2d	se H2c ha aiutato sulle frasi lunghe	media pesata dei token decisa dalla testa (fase 1b, 09 §4)
H-PAIR	se l'insieme completo su HoVer resta sotto 0,65, o l'F1 cala di oltre 5 punti con frasi lontane	punteggio per coppie di frasi (08 §3 e §6.4)
H6	se gli obiettivi non sono raggiunti dopo tutto il resto	fase 2 (09 §5), con peso dell'evidenza 0,1 / 0,3 / 1

elephant-1 e gli altri modelli

Nome	Che cosa
E0	elephant-1: estrazione e configurazione migliore di KnowLine; controllo rapido dei layer con la sonda (15 §3), perché possono essere diversi
X0	gli altri decision model: prima quelli con il codice degli autori in transformers (Jeffan-4B, d3-flash, OneJev, Nimble), poi StartLux (maschera da ricostruire), pplx e Clef (testa propria)
E6	fase 2, solo se su KnowLine ha dato un guadagno chiaro e il modello è sotto gli obiettivi
L1	latenza per ogni modello rilasciato (10 §5)

Controlli finali

Nome	Che cosa
C1	3 semi della testa finale per modello: media e variabilità
C2	sufficienza e necessità su tutti i set di test
C3	testa contro E-ABL: quanto ci si avvicina con un forward a un metodo da centinaia di forward
C4	confronto fra i modelli sulle righe giudicate supportate da tutti

Registro

`data/evidence/runs/registry.md`, una riga per prova (scritta in automatico dallo script di training, tranne la decisione):

Nome	Modello	Fase	Modifica	F1	Insieme completo	F1 ≥ 10 frasi	F1 > 1.024 token	Negativi accesi	Sufficienza	Δ verdetto	Δ latenza	Decisione
E-LEX	—	—	—							—	—	riferimento
H0	KnowLine-4B	1	—							0		

Passo 12 — Server e rilascio

In breve

Un server (`evidence/serve.py`) che per ogni richiesta fa **un forward** e restituisce verdetto ed evidenza. Usa lo stesso codice della valutazione (divisione in frasi, rendering, hook, testa, selezione): quello che si è misurato è quello che gira.

1. Che cosa fa il server per ogni richiesta

```
richiesta {document, claim}
  1. divide il documento in frasi (evidence/sentences.py, lo stesso divisore del training)
  2. prompt con il preambolo (codice degli autori) e token di ogni frase (evidence/render.py)
  3. UN forward con gli hook: punteggi "sì"/"no", vettori delle frasi (fine e media, 4 layer), attenzione, q
  4. P(supportato); se non supera la soglia → risposta senza evidenza
  5. testa → s[k], s0 → frasi sopra la soglia (almeno 1, al massimo 5) → posizioni nel documento originale
```

```
def answer(document, claim):
    spans = sentences.split(document)
    ex = render(document, claim, spans, preamble=DECISION["preamble"])          # input_ids, answer_pos,
↪   sent_tok_spans
    logits, feats = model.forward_sentences(ex)                                 # un forward con gli hook
    p = p_supported(logits, DECISION["temperature"], DECISION["take"])
    if p <= DECISION["threshold"]:
        return {"p_supported": p, "supported": False, "evidence": []}
    s, s0, _ = head(**feats)
    shown = select(s[0].tolist(), float(s0[0]), **DECISION["selection"])
    return {"p_supported": p, "supported": True,
            "evidence": [{"sentence": k, "start": spans[k][0], "end": spans[k][1],
                          "score": float(torch.sigmoid(s[0, k] - s0[0]))} for k in shown]}
```

`start` / `end` si riferiscono al documento come l'ha inviato il client, non al prompt.

2. Come gira ogni modello

Il server ha bisogno degli **stati interni** del forward, non solo della risposta: il forward si fa in transformers, con il codice degli autori e gli hook.

Modello	Forward nel server
KnowLine-4B	front end degli autori (`knowline_server.py`) con backend transformers, più gli hook
elephant-1	motore `qev` in-process (legge la maschera a blocchi da `modeling_overrides`, solo da una cartella locale)
Jeffan-4B, d3-flash, pplx-decider-flash, Clef-Flash	il codice degli autori in transformers
OneJev1.6-9B, Nimble-9B-v3	transformers con il prompt degli autori: i loro runtime (Flide, motore Nimble) non espongono gli stati; il verdetto si verifica contro il loro runtime
StartLux 9B/4B	transformers con la maschera bidirezionale ricostruita: il server degli autori è vLLM e non espone gli stati; il verdetto si verifica contro il loro server

Regole:

- servono `flash-linear-attention` e `causal-conv1d` (layer DeltaNet), altrimenti il forward è molto più lento;
- il riassunto delle frasi si fa sulla GPU negli hook: escono n × 4 × D numeri, mai i vettori di tutti i token;
- per KnowLine, nelle richieste lette due volte, si usano le frasi della seconda copia;
- richieste raggruppate per lunghezza simile;
- documenti oltre il limite di token: errore esplicito (`document_too_long`), mai troncati.

3. La cartella di rilascio

`releases/<modello>-fc-ev-<versione>/`:

File	Contenuto
pesi del verdetto	quelli del fine-tuning binario (o riaddestrati con il preambolo, o con la fase 2)
`evidence_head.safetensors`	pesi della testa (15 MB per KnowLine-4B, 19,6 MB per elephant-1)
`sentences/`	il divisore: `sentences.py` e i parametri inglesi di `punkt_tab` (§4)

File	Contenuto
`decision.json`	tutte le impostazioni (sotto)
`frozen.json`	hash di pesi, testa, `sentences.py`, `punkt_tab`, `render.py`, `escape.py`, `question.json`, split e mix
`report.md`	risultati dei test e latenza (10 §7)
`MODEL_CARD.md`	descrizione per chi usa il modello

```
{"model": "knowline-4b-fc-ev-v1",
 "question_id": "…", "take": "…",
 "preamble": "Claim under review: {claim}",
 "temperature": 1.0, "threshold": 0.50,
 "evidence": {"mix_layers": [19, 23, 27, 31], "attention_layers": [3, 7, 11, 15, 19, 23, 27, 31],
              "features": ["end", "mean", "att"], "head_config": "configs/<id>.json"},
 "selection": {"delta": 0.0, "max_shown": 5, "min_shown": 1},
 "segmenter": {"file": "sentences/sentences.py", "sha256": "<…>", "punkt_tab_sha256": "<…>", "nltk": "3.10.4"},
 "max_prompt_tokens": 32768}
```

(Valori di esempio.) Se cambia il divisore, il preambolo o i layer, la testa va riaddestrata.

4. Il divisore in frasi va rilasciato con il modello

La testa lavora su frasi: in produzione il documento va diviso **esattamente** come in training. Il risultato del divisore dipende da tre cose fuori dal nostro codice: la versione di NLTK, i file `punkt_tab` (in `~/nltk_data`, con il problema dei permessi) e la macchina. Per non dipenderne:

1. copiare nella cartella di rilascio (e nella repo, in `evidence/punkt/`) i parametri inglesi di `punkt_tab` (pochi file, licenza Apache 2.0) e caricarli da lì;
2. fissare la versione di NLTK nei requisiti;
3. registrare gli hash in `frozen.json` e verificarli all'avvio del server.

Così il divisore è identico ovunque, senza `nltk.download` e senza permessi da sistemare. La divisione costa qualche millisecondo di CPU, trascurabile rispetto al forward.

5. Controlli prima del rilascio

Controllo	Deve dare
Verdetto del server = verdetto in valutazione (200 righe)	differenza media di P < 0,005
Evidenza del server = evidenza in valutazione (200 righe)	stesse frasi in almeno 199 righe
Frasi del server = frasi del training (stessi documenti)	identiche, su 1.000 documenti
Richieste in batch con lunghezze diverse	stesse frasi che una alla volta
Documenti con `"`, `\`, a capo, `<`, emoji, testo non latino	il testo fra `start` ed `end` è la frase giusta
Latenza	entro il limite (10 §5)

6. Congelamento, test, model card

1. Scrivere `frozen.json`.
2. Eseguire **una sola volta** i set di test e la latenza; scrivere `report.md`.
3. Model card: verifica documento–claim in inglese; evidenzia frasi **solo** quando il claim è supportato; non dice cosa manca né cosa contraddice; il prompt contiene una riga col claim prima del documento; documenti fino a 32.768 token; l'evidenza è a livello di frase, con il divisore incluso nel rilascio.
4. Licenze: quelle dei pesi di partenza (MIT per elephant-1; Apache 2.0 per KnowLine-4B, Jeffan-4B, d3-flash, OneJev, pplx-decider-flash, Clef-Flash; **CC BY-NC per StartLux e Nimble**, quindi derivati non commerciali). Dati della testa: HoVer CC BY-SA; SciFact e WiCE con le loro licenze.

Checklist e problemi frequenti

1. Checklist per passo

Passo 1 — Modelli e hook per frase

- ☐ `flash-linear-attention` e `causal-conv1d` installati e importabili; `punkt_tab` disponibile e `~/nltk_data` privata.
- ☐ `SentenceTap` e `sdpa_capture` funzionanti per KnowLine-4B; test superati (verdetto, layer, attenzione contro "eager", maschera, riassunti delle frasi, memoria).

Passo 2 — Dati

- ☐ Dataset costruiti con `--hover-db-fallback`; allineamento ≥ 90% per fonte.
- ☐ 30 positivi per fonte letti con le frasi marcate.

Passo 3 — Dati generati (facoltativo per le prime prove)

- ☐ Articoli con posizioni note; ablazione con conferma del giudice ≥ 80% e accordo con elephant-1 ≥ 0,7; evidenza spostata senza riferimenti rotti.

Passo 4 — Split e mix

- ☐ Split per `doc_id`; WiCE con `aggrefact_overlap` solo in test; dev e test di HoVer solo `exact`.
- ☐ Mix `ev_head` con i vincoli rispettati.

Passo 5 — Preambolo

- ☐ Testo scelto; verdetto riusato o riaddestrato; `question.json` aggiornato.

Passo 6 — Dalle frasi ai token, estrazione

- ☐ `escape_with_map`, `token_char_spans`, `sentence_token_spans` testati; frasi decodificate = frasi (200 su 200).
- ☐ Vettori di KnowLine-4B salvati per il mix e i set dev, con `meta.json`.

Passo 7 — Baseline

- ☐ E-RAND, E-LEX, E-EMB, E-ATT su `dev_evid_select`; E-ABL su 300 righe per fonte.

Passo 8 — Testa e loss

- ☐ `tests/test_evidence_head.py` passa (forme, invarianza al riempimento, esempi numerici).

Passo 9 — Training

- ☐ Impara a memoria 64 righe; H0 riproducibile; regola di selezione scelta e confermata.

Passo 10 — Valutazione

- ☐ Tutte le metriche, le prove di comportamento e la latenza A/B/C.

Passo 11 — Esperimenti

- ☐ Esperimenti principali e controlli su KnowLine-4B; configurazione ripetuta su elephant-1 e sugli altri modelli; registro compilato.

Passo 12 — Rilascio

- ☐ Divisore e `punkt_tab` inclusi nel rilascio; controlli del server superati; `frozen.json` prima dei test; test una volta; model card.

2. Se succede...

Problema	Prima cosa da controllare	Poi
La testa non impara (F1 vicino al caso)	le posizioni dei token di ogni frase (passo 6, test)	le impostazioni dei vettori salvati (`meta.json`)

Problema	Prima cosa da controllare	Poi
La testa fa come E-LEX: sbaglia frasi riscritte e frasi ponte	quante righe HoVer e generate ci sono nel mix	più negativi con frasi simili al claim; H-MIL con peso 1
Bene su SciFact, male su HoVer e con frasi lontane	H3a: il transformer fra le frasi serve?	più righe spostate; H-PAIR; 4 layer di transformer
Male quando l'evidenza è lontana dall'inizio del documento	quante righe lontane ci sono nel mix	attenzione come feature (H2b); fase 2
Mostra troppe frasi	la regola di selezione (δ) e i negativi nel training	ω più alto
Non mostra quasi nulla sui positivi incerti	peso di `mil_neg`	abbassarlo
Mostra spesso la prima frase o il titolo	dove stanno le frasi gold nel mix	H9 (senza posizione)
Precisione bassa ma sufficienza alta	leggere a mano le frasi "sbagliate": spesso sono evidenza valida non annotata	ω più basso per quella fonte
Le etichette ricavate col modello peggiorano (H8)	accordo del giudice e fra KnowLine ed elephant-1	solo le righe concordi, o `mil_pos` al posto dell'ablazione
Risultati molto diversi fra semi	se la testa dipende da poche teste di attenzione (pesi di `W_a`)	dropout sull'attenzione più alto, o toglierla
Il verdetto cambia nelle fasi 1a/1b	non deve succedere: il prompt è diverso da quello del passo 5	test di identità del prompt
Frasi diverse fra training e server	versione di NLTK o file `punkt_tab` diversi	caricare `punkt_tab` dalla repo; hash in `frozen.json`
Troppo lento	tempo della testa e degli hook misurati a parte	meno layer, togliere media o attenzione, 1 layer di transformer
Spazio su disco insufficiente	quanti layer si salvano	solo i layer con peso, o i migliori secondo la sonda

Una strada da esplorare: insegnare alla testa quanto ogni frase regge il verdetto

Il problema

Le frasi di evidenza annotate vengono da Wikipedia, pagine web e abstract biomedici. In produzione arrivano altri testi: riassunti, risposte di sistemi RAG, notizie, documenti tecnici. Su questi abbiamo il verdetto ma non sappiamo quali frasi sono l'evidenza, quindi la testa non impara a lavorarci.

Le etichette ricavate col modello (passo 03 §2) aiutano solo in parte: dicono quali frasi bastano, ma non quanto conta ciascuna, e costano una ricerca per riga.

L'idea

Su **qualunque** riga supportata, di qualunque tipo di testo, si può misurare senza annotazioni quanto ogni frase regge il verdetto: si toglie la frase dal documento e si guarda di quanto scende il punteggio del verdetto.

```
z           = margine del verdetto (sì − no) sul documento intero
z senza k   = margine col documento senza la frase k            (un forward per frase)
importanza_k = z − z senza k                                    quanto la frase k regge il verdetto
```

La testa impara a prevedere questo ordinamento in un solo forward. Così impara, sui testi di produzione, quali frasi reggono davvero il verdetto, che è anche la metrica di sufficienza.

Se la stessa informazione è ripetuta in due frasi, togliendone una il verdetto non cambia, e entrambe

sembrano poco importanti. Per questo si tiene anche il gruppo trovato con l'ablazione del passo 3, che gestisce questo caso.

Come farlo

1. **Righe**: positivi senza evidenza annotata da testi di produzione o simili (riassunti, risposte RAG dei dati di training del fine-tuning binario), più HoVer e SciFact per confrontare con le annotazioni. Niente righe di test né pagine di LLM-AggreFact.
2. **Calcolo con KnowLine-4B** (`evidence/label_counterfactual.py`): per ogni riga, un forward per frase tolta. KnowLine è causale, quindi i prompt sono uguali fino alla frase tolta e si riusa il calcolo della parte iniziale comune (con un modello bidirezionale non si potrebbe). Documenti fino a 80 frasi; oltre, solo per le 50 frasi con punteggio più alto della testa della fase 1a.
3. **Prima verifica, senza training**: su `dev_evid_select`, si usa l'importanza come punteggio delle frasi e si confronta con le annotazioni. Se non fa meglio di E-LEX, il segnale è troppo rumoroso e ci si ferma.
4. **Accordo fra modelli**: su 300 righe si rifà il calcolo con elephant-1. Se gli ordinamenti sono molto diversi, per gli altri modelli si ricalcola col modello stesso.
5. **Training** (fase 1a, esperimento H-CF): alla loss della testa si aggiunge un termine che avvicina l'ordinamento dei punteggi della testa a quello dell'importanza:

   ```
   L_cf = KL( softmax(importanza / T) ‖ softmax(s / T') )      sulle frasi della riga
   L    = loss ZLPR (con il gruppo dell'ablazione come evidenza, peso 0,5)  +  η · L_cf        η ∈ {0,3; 1}
   ```

   T si sceglie in modo che le 3 frasi più importanti abbiano ~80% della massa.
6. **Valutazione** su tre set:
   - `dev_evid_select` (annotato): non deve peggiorare di più di 1 punto di F1;
   - `test_evid_ood` (WiCE);
   - un nuovo piccolo set di 200 righe di riassunti e risposte RAG, con evidenza indicata da un giudice LLM e controllata a mano.

Quando tenerlo

Rispetto alla configurazione migliore senza questa aggiunta:

- sufficienza almeno +3 punti su tutti i set;
- F1 almeno +2 punti sul set di riassunti e RAG;
- F1 su `dev_evid_select` non più di 1 punto sotto.

Rischi

Rischio	Cosa fare
L'importanza segue le scorciatoie del modello (per esempio frasi con parole del claim che il modello sopravvaluta)	la verifica del punto 3; tenere sempre le annotazioni dove ci sono
Informazione ripetuta: importanza bassa per tutte le copie	il gruppo dell'ablazione come evidenza
Costo: un forward per frase	KnowLine-4B, riuso della parte comune, limite di 80 frasi
Se il modello viene riaddestrato (fase 2), l'importanza calcolata prima non vale più	usarla solo col modello congelato, o ricalcolarla

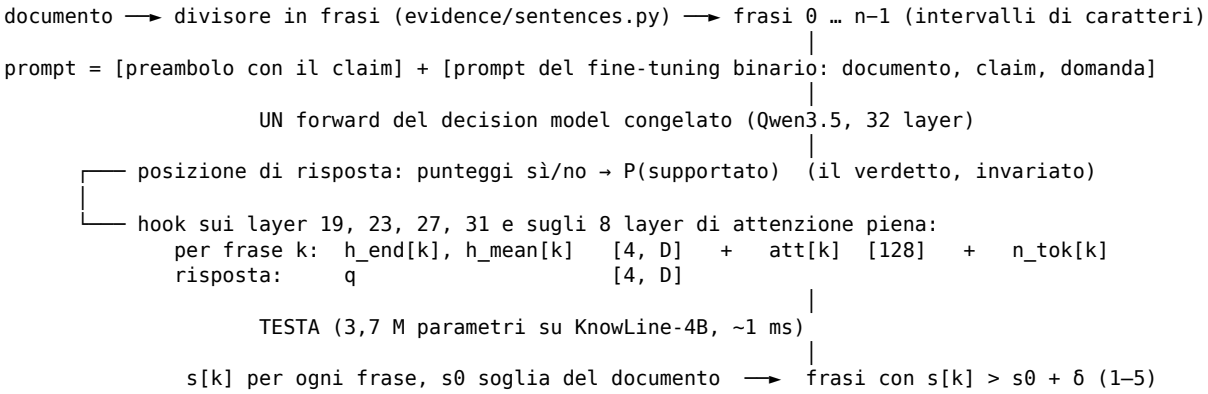
Perché vale la pena

È l'unico modo, fra quelli del piano, di avere un segnale di evidenza sui testi di produzione senza annotarli a mano, ed è legato direttamente a ciò che interessa all'utente: che le frasi mostrate bastino a reggere il verdetto. Il costo è tutto prima del rilascio e riusa codice già previsto; in produzione non cambia nulla.

Riepilogo — Architettura, layer da leggere, loss

Questo file raccoglie in un posto solo le tre scelte tecniche che definiscono la testa di evidenza per frase: com'è fatta, da quali layer del decision model legge e perché, con quale loss si addestra. I dettagli operativi sono nei passi 01, 06, 08 e 09.

1. Il quadro



Tre principi:

1. **Un solo forward**: la testa usa vettori che il modello calcola comunque; nessun token generato, nessun forward in più.
2. **Il verdetto non cambia**: il modello è congelato; si addestra solo la testa.
3. **Le frasi si vedono fra loro**: un transformer non causale sulle frasi risolve l'evidenza sparsa, che il modello causale da solo non può risolvere (la frase 1 non vede la frase 40).

2. L'architettura in sintesi

Stadio	Operazione	Forma in uscita	Parametri (KnowLine-4B)	Perché
A. Miscela dei layer	`LN(Σ_l softmax(α)_l · v[l])`, stessi α per `h_end`, `h_mean`, `q`	[n, D]	5.124	combina i 4 layer con pesi appresi; la LN uniforma le scale
B. Vettore della frase	`W_h [h_end ; h_mean] + W_a [att ; log n_tok] + P[k] + T_frase`	[n, 256]	1.475.840	contenuto della frase (fine e media), dove ha guardato il modello, posizione
B'. Vettore di risposta	`W_q q + T_risposta`	[256]	655.616	il punto di vista del verdetto

Stadio	Operazione	Forma in uscita	Parametri (KnowLine-4B)	Perché
C. Comunicazione	transformer non causale, 2 layer, 4 teste, FFN 1.024, su `[x0, x[0…n−1]]`	[n+1, 256]	1.579.520	ogni frase vede tutte le altre e la risposta: evidenza sparsa
D. Punteggi	`w_o · X` per frasi e per `x0`	s [n], s0 []	257	un'unica scala per punteggi e soglia
(H-PAIR)	bilineare a rango 64 sulle coppie	[n, n]	32.769	solo se l'evidenza multi-frase resta difficile
Totale			**3.716.357** (elephant-1: 4.899.077)	

Tempo misurato: 0,67 ms con 46 frasi, 1,55 ms con 500 (RTX 5090 Laptop). Dettagli stadio per stadio: 08 §3.

Ingressi per frase: **ultimo token** (riassume la frase letta fin lì, vede anche il contesto precedente) e **media dei token** (pesa tutta la frase, non dipende dalla punteggiatura finale). Insieme coprono i due modi in cui un'informazione può essere "dentro" una frase.

3. Da quali layer leggere, perché, e come sceglierli

3.1 Che cosa offre Qwen3.5

Layer	Tipo	Che cosa fa
0, 1, 2, 4, 5, 6, … (24 in tutto)	Gated DeltaNet	attenzione lineare, sempre causale: aggiornano un riassunto del testo letto, con memoria che decade
3, 7, 11, 15, 19, 23, 27, 31 (8)	attenzione piena	ogni token guarda esplicitamente tutti gli altri (solo i precedenti nei modelli causali, tutti nei bidirezionali): è qui che parti lontane del testo si collegano
31	ultimo layer	prepara la predizione del token successivo, cioè la lettera della risposta

3.2 Il punto di partenza: 19, 23, 27, 31

Gli ultimi quattro layer ad attenzione piena. Il ragionamento:

- **dopo un layer ad attenzione piena** il vettore di una frase ha appena "visto" tutto il testo che può vedere (il claim del preambolo, le altre frasi). I layer DeltaNet in mezzo portano informazione solo in avanti e con memoria che decade: leggere subito dopo un layer pieno è leggere la rappresentazione più aggiornata;
- **nella metà alta della rete**: nei primi layer i vettori rappresentano soprattutto le parole e la sintassi locale, non ancora il rapporto fra la frase e il claim; nella metà alta compaiono le rappresentazioni del contenuto e del confronto con il claim;

- **non solo l'ultimo**: l'ultimo layer è specializzato a produrre la risposta nella posizione di risposta; nelle altre posizioni può aver già "buttato via" informazione che non serve a predire il token successivo. I layer pieni precedenti (19, 23, 27) conservano di più il contenuto della frase;
- **quattro, mescolati con pesi appresi**: non si sa a priori quale layer serve di più, e può cambiare da modello a modello. La miscela lascia decidere al training, con un costo minimo (4 parametri) e uno spazio su disco proporzionale al numero di layer.

Sono ipotesi ragionevoli, non misure: vanno verificate con i due metodi seguenti prima di fissare i layer.

3.3 Come sceglierli davvero: la sonda per layer (prima del training)

Su un campione piccolo si estraggono i vettori delle frasi in **tutti** i 32 layer, e per ogni layer si addestra una regressione logistica (una "sonda") che deve riconoscere le frasi di evidenza dal solo vettore della frase. Un layer in cui la sonda funziona bene contiene già l'informazione che serve alla testa.

1. **Campione**: 500 righe positive del training (HoVer e SciFact) e 200 del dev. Spazio: ~12 GB su KnowLine-4B (700 righe × ~52 frasi × 32 layer × 2 vettori).
2. **Estrazione**: lo stesso forward del passo 6, con gli hook su tutti i 32 layer (`h_end`, `h_mean`); con il preambolo.
3. **Sonda**: per ogni layer, regressione logistica su `[h_end ; h_mean]` normalizzato, classi bilanciate; si misura l'AUPRC per frase sul dev.
4. **Scelta**: i 4 layer con AUPRC più alta, preferendo a parità i layer ad attenzione piena (le feature di attenzione si leggono comunque da lì). Se i migliori sono ancora 19–31, si conferma il punto di partenza.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score


def probe_layers(train, dev, layers):
    """train, dev: liste di righe con, per ogni layer l, h_end[l] e h_mean[l] ([n, D] numpy) e gold [n].
    Per ogni layer: regressione logistica su [h_end ; h_mean] (vettori normalizzati), AUPRC sul dev.
    -> {layer: AUPRC}, ordinato dal migliore."""
    def xy(rows, l):
        x = np.concatenate([np.concatenate([r["h_end"][l], r["h_mean"][l]], 1) for r in rows])
        x = x / (np.linalg.norm(x, axis=1, keepdims=True) + 1e-6)
        return x, np.concatenate([r["gold"] for r in rows])
    out = {}
    for l in layers:
        xt, yt = xy(train, l)
        xd, yd = xy(dev, l)
        clf = LogisticRegression(max_iter=2000, C=1.0, class_weight="balanced").fit(xt, yt)
        out[l] = float(average_precision_score(yd, clf.decision_function(xd)))
    return dict(sorted(out.items(), key=lambda kv: -kv[1]))
```

Provato su dati finti (un layer con il segnale e uno quasi senza): la sonda separa nettamente i due (AUPRC 0,87 contro 0,08).

Varianti utili della stessa sonda:

- con q concatenato (la frase letta "insieme" al verdetto): dice se la posizione di risposta aggiunge informazione;
- solo `h_end` o solo `h_mean`: dice se servono entrambi (esperimento H2c);
- sui modelli bidirezionali (elephant-1): i layer migliori possono essere diversi, perché anche le frasi iniziali vedono il claim nei layer pieni.

3.4 Come sceglierli davvero: i pesi della miscela (dopo il training)

Dopo il primo training con 4 layer (esperimento H2a), si guardano i pesi `softmax(α)`:

- un layer con meno del 10% si toglie (metà spazio su disco se ne restano 2, meno costo negli hook);
- se un layer prende quasi tutto il peso, basta quello;

- si riaddestra con i layer rimasti e si tiene la versione più semplice se l'F1 non scende di più di 1 punto (regola degli esperimenti, 11).

3.5 Le feature di attenzione: da quali layer e teste

L'attenzione esiste solo negli 8 layer pieni: si leggono tutti (8 × 16 = 128 numeri per frase), e il peso di ciascuno lo impara `W_a`. Per sapere quali contano, la baseline E-ATT (07 §2) misura l'AUPRC di ogni coppia (layer, testa) sul dev: se poche coppie portano quasi tutto il segnale, si può ridurre a quelle. Il dropout per frase sulle feature di attenzione evita che la testa dipenda da una sola testa di attenzione.

3.6 Riepilogo della procedura

Quando	Che cosa	Risultato
prima del training	sonda su tutti i 32 layer, 500 righe	i 4 layer candidati (atteso: 19, 23, 27, 31)
prima del training	E-ATT per layer e testa	quanto segnale c'è nell'attenzione
H0 → H2a	ultimo layer contro i 4 candidati	quanto guadagna la miscela
dopo H2a	pesi α	togliere i layer sotto il 10%
altri modelli	sonda veloce (200 righe)	conferma o cambio dei layer per quel modello

4. La loss

4.1 Che cosa deve insegnare

1. le frasi di evidenza devono stare **sopra** la soglia del documento, le altre **sotto**;
2. sui claim **non supportati** nessuna frase deve stare sopra la soglia (così la testa non evidenzia una frase solo perché somiglia al claim);
3. con annotazioni incomplete, una frase valida non segnata non deve essere punita troppo;
4. il numero di frasi mostrate deve adattarsi al documento (1 per un fatto semplice, 3–4 per una catena multi-hop).

4.2 La loss principale: ZLPR con soglia del documento

Con `G` = frasi gold, `ω` = peso delle frasi non gold, `s0` = soglia prodotta dalla testa per quel documento:

```
supervised (positivi con evidenza):   log(1 + Σ_{i ∉ G} ω · e^(s_i − s0))   +   log(1 + Σ_{j ∈ G} e^(s0 − s_j))
                                     └─ cresce se una frase non gold sta sopra s0 ┘   └─ cresce se una frase gold
 ↪  sta sotto s0 ┘
mil_neg   (negativi):                 0,3 · log(1 + Σ_i e^(s_i − s0))                 nessuna frase sopra s0
mil_pos   (positivi senza evidenza):  0,3 · [ log(1 + e^(s0 − LSE_j s_j)) + 0,1 · media σ(s_i − s0) ]     almeno
 ↪  una, ma poche
none:                                 0
```

Requisito (§4.1)	Come lo soddisfa
1. gold sopra, altre sotto	i due termini di `supervised`
2. niente sui negativi	`mil_neg`: lo stesso primo termine, con tutte le frasi
3. annotazioni incomplete	ω < 1 (0,5 HoVer e SciFact, 0,3 WiCE) sulle frasi non gold
4. numero adattivo	la soglia `s0` è prodotta per ogni documento; la regola in uso è `s[k] > s0`

Esempio (test): 40 frasi, gold = {1, 40}, `s0 = 0`. Con la frase 40 a −0,5 (sotto soglia) e una frase simile al claim a 0,5 (sopra), la loss è **2,526**; dopo aver portato la frase 40 a 1,5 e la frase simile a −1, scende a **1,473**.

4.3 La loss di riferimento (H0)

BCE pesata per frase (peso α ≈ 10 sulle frasi gold, ω sulle altre) più un termine di ordinamento (ogni gold sopra la migliore non gold di un margine 1), con una soglia fissa τ_e scelta sul dev. Serve da punto di partenza e da confronto: l'esperimento H-LOSS dice se la soglia del documento fa meglio.

4.4 Termine facoltativo per le coppie (H-PAIR)

Sulle righe con 2 o più frasi gold: punteggio di coppia alto fra frasi gold, basso fra una gold e le 5 non gold più vicine; peso 0,2.

4.5 Pesi e come si combinano

Peso	Valore	Dove
ω (frasi non gold)	0,5 HoVer e SciFact; 0,3 WiCE; 1,0 dati generati	nel primo termine di `supervised`
peso della riga	1,0; 0,5 per HoVer `db_split` e per le etichette ricavate col modello	moltiplica la loss della riga
μ_neg	0,3 (H-MIL prova 0, 0,3, 1)	`mil_neg`
μ_pos, ρ	0,3, 0,1	`mil_pos` (solo H-MIL)
λ (fase 2)	0,3	loss totale = loss del verdetto + λ · loss di evidenza

La loss del batch è la somma delle loss delle righe pesate, divisa per la somma dei pesi delle righe che hanno un termine di evidenza.

4.6 Il rischio da sorvegliare

`x0` conosce il verdetto, quindi con `mil_neg` la testa può imparare "se il verdetto è no, alza la soglia". È coerente con l'obiettivo, ma sui positivi incerti (P appena sopra la soglia del verdetto) la testa può non mostrare nulla. Si controlla la recall in quella fascia a ogni epoca; se cala, si abbassa μ_neg. In produzione resta comunque la regola "almeno una frase se il verdetto è supportato".